Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

M3dmet functions

Synopsis

The functions prefixed with M3dmet make up the Aurora Imaging Library 3D Metrology module.

Functions

M3dmetAlloc
M3dmetAllocResult
M3dmetControl
M3dmetControlDraw
M3dmetCopy
M3dmetCopyResult
M3dmetDistance
M3dmetDistanceEx
M3dmetDraw3d
M3dmetFeature
M3dmetFeatureEx
M3dmetFit
M3dmetFree
M3dmetGetResult
M3dmetInquire
M3dmetInquireDraw
M3dmetRestore
M3dmetSave
M3dmetStat
M3dmetStream
M3dmetVolume
M3dmetVolumeEx

M3dmetAlloc

Synopsis

Allocate a 3D metrology context.

Syntax

```
AIL_ID M3dmetAlloc(
    AIL_ID SysId,             //in
    AIL_INT64 ContextType,    //in
    AIL_INT64 ControlFlag,    //in
    AIL_ID *Context3dmetIdPtr //out
)
```

Description

This function allocates a 3D metrology context on the specified system. A 3D metrology context contains information needed to perform an M3dmetDistanceEx(), M3dmetFit(), M3dmetStat(), or M3dmetVolumeEx() operation. You can also use **M3dmetAlloc**() to allocate a draw 3D metrology context for drawing results using M3dmetDraw3d().

When the 3D metrology context is no longer required, release it using M3dmetFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the 3D metrology context's lifetime and you must not manually free it.

Parameters

SysId

Specifies the system on which to allocate the 3D metrology context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the type of 3D metrology context to allocate. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DISTANCE_CONTEXT	Specifies to allocate a distance 3D metrology context to use with M3dmetDistanceEx().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_3D_CONTEXT	Specifies to allocate a draw 3D metrology context to use with M3dmetDraw3d().	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_CONTEXT	Specifies to allocate a fit 3D metrology context to use with M3dmetFit().	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_CONTEXT	Allocates a statistics 3D metrology context to use with M3dmetStat().	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_CONTEXT	Specifies to allocate a volume 3D metrology context to use with M3dmetVolumeEx().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Context3dmetIdPtr

Specifies the address of the variable in which to write the 3D metrology context identifier or specifies the data type that the function should use to

return the 3D metrology context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D metrology context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D metrology context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DMET_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D metrology context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the distance 3D metrology context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated distance 3D metrology context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the fit 3D metrology context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated fit 3D metrology context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics 3D metrology context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated statistics 3D metrology context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D metrology context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DMET_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetAllocResult

Synopsis

Allocates a 3D metrology result buffer.

Syntax

```
AIL_ID M3dmetAllocResult(
    AIL_ID SysId,            //in
    AIL_INT64 ResultType,    //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *Result3dmetIdPtr //out
)
```

Description

This function allocates a 3D metrology result buffer on the specified system, to store results obtained from a 3D metrology operation. You can specify a fit 3D metrology result buffer, for use with M3dmetFit(), a statistics 3D metrology result buffer, for use with M3dmetStat(), or a calculate 3D metrology result buffer, for use with M3dmetDistanceEx() M3dmetFeatureEx(), or M3dmetVolumeEx().

When the 3D metrology result buffer is no longer required, release it using M3dmetFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the 3D metrology result buffer's lifetime and you must not manually free it.

Parameters

SysId

Specifies the identifier of the system on which to allocate the 3D metrology result buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of 3D metrology result buffer to allocate.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CALCULATE_RESULT	Specifies to allocate a calculate 3D metrology result buffer, used for a M3dmetDistanceEx(), M3dmetFeatureEx() or M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_RESULT	Specifies to allocate a fit 3D metrology result buffer, used for a M3dmetFit() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_RESULT	Specifies to allocate a statistics 3D metrology result buffer, used for a M3dmetStat() operation.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Result3dmetIdPtr

Specifies the address of the variable in which to write the 3D metrology result buffer identifier or specifies the data type that the function should use to return the 3D metrology result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D metrology result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D metrology result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DMET_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D metrology result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the calculate 3D metrology result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated calculate 3D metrology result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the fit 3D metrology result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated fit 3D metrology result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics 3D metrology result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated statistics 3D metrology result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D metrology result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DMET_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetControl

Synopsis

Control a setting for a 3D metrology context or result buffer.

Syntax

```
void M3dmetControl(
    AIL_ID ContextOrResult3dmetId, //out
    AIL_INT64 ControlType,         //in
    AIL_DOUBLE ControlValue        //in
)
```

Description

This function allows you to control a setting of a 3D metrology context or result buffer. You can typically inquire these settings using M3dmetInquire().

To control draw 3D metrology settings for M3dmetDraw3d(), use M3dmetControlDraw() instead.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **M3dmetControl()** internally calls the *AIL_DOUBLE* version of this function (M3dmetControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (M3dmetControlInt64()).

Parameters

ContextOrResult3dmetId

Specifies the identifier of the 3D metrology context or result buffer to control. The 3D metrology context or result buffer must have been previously allocated on the required system using M3dmetAlloc() or M3dmetAllocResult(), respectively.

ControlType

Specifies the setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the value needed for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- To control a statistics 3D metrology context
- To control a fit 3D metrology context
- To control a volume 3D metrology context
- To control a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx operation
- To control a distance 3d metrology context

The following ControlType and ControlValue parameter settings can be specified for statistics 3D metrology contexts, and control which statistics M3dmetStat() calculates.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control a statistics 3D metrology context																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																	
M_STAT_MAX		Sets whether to calculate the maximum distance between the point cloud or depth map, and the specified reference object. INQ																
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies to calculate the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS		Sets whether to calculate the maximum absolute distance between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the maximum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the maximum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN		Sets whether to calculate the average distance between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the average distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the average distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN_ABS		Sets whether to calculate the average absolute distance between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the average absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the average absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN		Sets whether to calculate the minimum distance between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the minimum distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the minimum distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS		Sets whether to calculate the minimum absolute distance between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the minimum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the minimum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_NUMBER		Sets whether to record the number of points that meet the condition specified when calling M3dmetStat() (using the Condition, CondLow and CondHigh parameters). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to record the number of points that meet the specified condition.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies to record the number of points that meet the specified condition.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_RMS		Sets whether to calculate the root-mean-square (RMS) error between the point cloud or depth map, and the specified reference object. Aurora Imaging Library calculates the RMS error using the following formula: INQ $$\text{RMS error} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}[(\bar{x}_i - x_i)^2 + (\bar{y}_i - y_i)^2 + (\bar{z}_i - z_i)^2]}$$	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the RMS error.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the RMS error.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION		Sets whether to calculate the standard deviation of all the distances calculated between the point cloud or depth map, and the specified reference object. Aurora Imaging Library calculates the standard deviation using the following formula: INQ $$s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}[(\bar{x}_i - x_i)^2 + (\bar{y}_i - y_i)^2 + (\bar{z}_i - z_i)^2]}$$	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM		Sets whether to calculate the sum of all the distances calculated between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the sum of all the distances.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the sum of all the distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS		Sets whether to calculate the sum of all the absolute distances calculated between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the sum of all absolute distances.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the sum of all absolute distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES		Sets whether to calculate the sum of squared distances between the point cloud or depth map, and the specified reference object. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the sum of squared distances.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the sum of squared distances.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and ControlValue parameter settings can be specified for a fit 3D metrology context, and control how M3dmetFit() performs the fit operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control a fit 3D metrology context

ControlType	ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ESTIMATION_MODE		Sets how to compute an initial fit estimate between the point cloud or depth map, and the specified reference object. INQ															
	M_DEFAULT	Same as M_RANDOM_SAMPLING.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FROM_GEOMETRY	Specifies that an initial fit estimate is determined using a geometry object that is copied into the fit 3D metrology context using M3dmetCopy() with M_ESTIMATE_GEOMETRY. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include cylinder, line, plane, and sphere. Note that fitting a 3D box geometry is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_SAMPLING	Specifies that an initial fit estimate is calculated using all available points in a point cloud or depth map. It is recommended that you only use this mode when there is a low percentage of outliers among the points of the point cloud or depth map.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RANDOM_SAMPLING	Specifies that an initial fit estimate is determined using a random sampling consensus (RANSAC) algorithm. This mode tries to find the best fit of a 3D geometry object to a point cloud by trying to fit the reference object to several random samplings of the point cloud or depth map, and uses the best fit (with the lowest root-mean-squared (RMS) error) as the best initial approximation. Random sampling consensus assumes that the best fit of the Aurora Imaging Library object will use the random sampling of points that contains the fewest outliers. Using an initial fit estimate that ignores outliers will prevent M3dmetFit() from fitting the Aurora Imaging Library object incorrectly. To optimize the likelihood of a good initial fit estimate, you can specify a value for M_EXPECTED_OUTLIER_PERCENTAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXPECTED_OUTLIER_PERCENTAGE		Sets the expected percentage of outliers among the points of the point cloud or depth map to be fitted. Note, this is only used for M3dmetFit() operations with OutlierDistance set to M_AUTO_VALUE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 40.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* < 100.0	Specifies the expected percentage of outliers among the points of the point cloud or depth map to be fitted.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_ITERATIONS_MAX		Sets the maximum number of iterations to use during the fit operation. If this value is set to 0, M3dmetFit() returns the result of the initial fit estimate, as specified using **M3dmetControl**() with M_ESTIMATION_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 5.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the maximum number of iterations to use during the fit operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_INLIER_AMOUNT_THRESHOLD	Sets a minimum number of inliers required for the fit operation to end. After each iteration of the fit operation, the total number of inliers and the RMS error are calculated. If the number of inliers exceeds M_INLIER_AMOUNT_THRESHOLD, and the average RMS error is less than M_RMS_ERROR_THRESHOLD, the fit operation will stop. A point is considered an inlier when its distance from the reference geometry is less than the value passed to the OutlierDistance parameter when M3dmetFit() is called. The status of the fit operation can be retrieved using M3dmetGetResult() with M_STATUS_FIT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that there is no minimum number of inliers.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the minimum number of inliers.	c		e	f	g	a		i		j	k	l	m	n	b
M_RMS_ERROR_THRESHOLD	Sets the maximum RMS error required for the fit operation to end. After each iteration of the fit operation, the average RMS error is calculated. If the average RMS error is less than M_RMS_ERROR_THRESHOLD, and the number of inliers exceeds M_INLIER_AMOUNT_THRESHOLD, the fit operation will stop. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum RMS error.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and ControlValue parameter settings can be specified for volume 3D metrology contexts, and control how M3dmetVolumeEx() performs the volume calculation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control a volume 3D metrology context																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_SAVE_VOLUME_INFO	Sets whether to save information from the volume computation for use with M3dmetCopyResult() or M3dmetDraw3d() operations. INQ															
M_DEFAULT	Same as M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the volume information is not saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the volume information is saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_MODE	Sets how to calculate the total volume, given the position of the source Aurora Imaging Library object relative to the reference object. When you specify M_COMPLETE, all volume results will be available to retrieve, copy, or draw. Otherwise, only the specified volume (for example, M_ABOVE) will be available. Use M_VOLUME_OUTPUT_MODE to set which result(s) to retrieve, copy, or draw. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. Same as M_COMPLETE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABOVE	Specifies that the total volume is equal only to the volume above the reference Aurora Imaging Library object. Any volume below the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPLETE	Specifies to calculate all volume results, which can then be retrieved, copied, or drawn, according to the specified M_VOLUME_OUTPUT_MODE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIFFERENCE	Specifies that the total volume is calculated as the volume above the reference Aurora Imaging Library object, minus the volume below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TOTAL	Specifies that the total volume is calculated as the sum of all volumes between the source and reference Aurora Imaging Library objects, whether above or below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNDER	Specifies that the total volume is equal only to the volume under the reference Aurora Imaging Library object. Any volume above the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and ControlValue parameter settings can be specified for a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx() operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx operation																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_VOLUME_OUTPUT_MODE		Sets the volume mode for which to retrieve, copy, or draw results, using M3dmetGetResult(), M3dmetCopyResult(), or M3dmetDraw3d(), respectively, when M_VOLUME_MODE is set to M_COMPLETE. If you have set M_VOLUME_MODE to a single volume statistic (that is, not M_COMPLETE), then you can leave this control type at its default setting (M_AUTO), which means that the output mode will automatically match that of M_VOLUME_MODE. If M_VOLUME_MODE is set to M_COMPLETE, any of the volume output modes are available. M_AUTO uses M_TOTAL. This control type is ignored when no reference object was specified and the volume was calculated for a mesh without holes, since the result is always the total volume. INQ															
	M_DEFAULT	Specifies the default value. Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABOVE	Specifies that the total volume is equal only to the volume above the reference Aurora Imaging Library object. Any volume below the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to match the M_VOLUME_MODE setting, unless M_VOLUME_MODE is set to M_COMPLETE, in which case M_TOTAL is used.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIFFERENCE	Specifies that the total volume is calculated as the volume above the reference Aurora Imaging Library object, minus the volume below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TOTAL	Specifies that the total volume is calculated as the sum of all volumes between the source and reference Aurora Imaging Library objects, whether above or below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNDER	Specifies that the total volume is equal only to the volume under the reference Aurora Imaging Library object. Any volume above the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and ControlValue parameter settings can be specified for distance 3D metrology contexts, and control how M3dmetDistanceEx() performs the distance calculation.

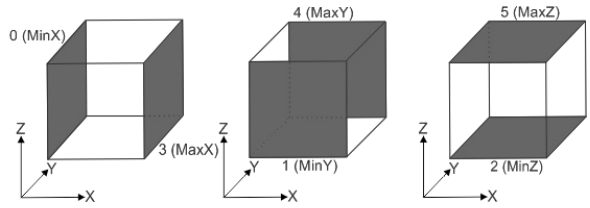
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control a distance 3d metrology context

ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_DISTANCE_METRIC_OPERATOR		Sets the operator applied to the distance from M_PAIRING_DISTANCE_METRIC. See M_PAIRING_TYPE for the supported settings for M_DISTANCE_METRIC_OPERATOR given the setting of M_PAIRING_TYPE, M_PAIRING_DISTANCE_METRIC, and the reference object. Note that if this control type can only be set to one setting, and you specify an unavailable setting, it will be ignored and the appropriate setting will be used instead. When more than one setting is available, and an unavailable setting is specified, an error is generated. INQ															
	M_DEFAULT	Same as M_ABSOLUTE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE	Specifies to use an absolute metric when calculating the distance. The returned distance is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED	Specifies to use a signed metric when calculating the distance. The returned distance can be either positive or negative.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SQUARED	Specifies to use a squared metric when calculating the distance. The returned value is the squared distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_NORMAL		Sets whether to compute the normal distance as the dot product between the paired source point's normal and the reference point's normal. To compute the normal distance, the source must be a point cloud container with an M_COMPONENT_NORMALS_AIL component. In addition, only the following M_PAIRING_TYPE settings are supported and must be used with the following reference objects: M_PAIRING_TYPE — Reference object M_DISTANCE_TO_SURFACE — Sphere, box, cylinder, plane, or a point cloud with an M_COMPONENT_MESH_AIL component. M_DISTANCE_TO_NEIGHBOR — Point cloud with an M_COMPONENT_MESH_AIL component. If these conditions are not met, an error is thrown. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not compute the normal distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to compute the normal distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRING_DISTANCE_MAX		Sets the maximum distance between a source point and a reference point to be paired. Distances larger than this are invalid. Note, this is only used when the reference object is a depth map or a point cloud. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies no maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b

M_PAIRING_DISTANCE_METRIC		Sets the metric used when choosing a source point's paired reference element. The paired reference element is that which is closest to the source point using the specified metric. Note, if the reference object is a depth map, only M_DIRECTION_Z is available. If another option is chosen, it will be ignored and M_DIRECTION_Z is still used. See M_PAIRING_TYPE for the supported settings for M_DISTANCE_METRIC_OPERATOR given the setting of M_PAIRING_TYPE, M_PAIRING_DISTANCE_METRIC, and the reference object. Note that if this control type can only be set to one setting, and you specify an unavailable setting, it will be ignored and the appropriate setting will be used instead. When more than one setting is available, and an unavailable setting is specified, an error is generated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to automatically choose the metric based on the reference object. If the reference object is a depth map, M_AUTO is the same as M_DIRECTION_Z. For all other reference objects, M_AUTO is the same as M_EUCLIDEAN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIRECTION_NORMAL	Specifies to use the distance in the normal direction of the source point. Note, the source container must be a point cloud container with an M_COMPONENT_NORMALS_AIL component or an error is thrown.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIRECTION_Z	Specifies to use the distance in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EUCLIDEAN	Specifies to use the Euclidean distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MANHATTAN	Specifies to use the Manhattan distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRING_PERPENDICULAR_DISTANCE_MAX		Sets the maximum perpendicular distance between a source point and a reference point to be paired. Distances greater than this are invalid. Note, this setting is only used when M_PAIRING_TYPE is set to M_DISTANCE_TO_NEIGHBOR, M_PAIRING_DISTANCE_METRIC is set to M_DIRECTION_NORMAL, and M_PAIRING_PERPENDICULAR_DISTANCE_MAX_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum perpendicular distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRING_PERPENDICULAR_DISTANCE_MAX_MODE		Sets whether an automatic value or a user-defined value is used for the maximum perpendicular distance between a source point and a reference point to be paired. Note, this setting is only used when M_PAIRING_TYPE is set to M_DISTANCE_TO_NEIGHBOR, M_PAIRING_DISTANCE_METRIC is set to M_DIRECTION_NORMAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies an automatic value. The automatic value is chosen based on the density of the point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies to use the user-defined value specified using M_PAIRING_PERPENDICULAR_DISTANCE_MAX.	c		e	f	g	a		i		j	k	l	m	n	b

M_PAIRING_TYPE		Sets against which part of the reference object to measure when performing an M3dmetDistanceEx() operation. All possible combinations of reference object, M_PAIRING_DISTANCE_METRIC, and M_DISTANCE_METRIC_OPERATOR are listed below each M_PAIRING_TYPE. When only one setting is available for M_PAIRING_DISTANCE_METRIC or M_DISTANCE_METRIC_OPERATOR, if an unavailable setting is specified, it will be ignored. For example, If you specify a box geometry with M_PAIRING_TYPE set to M_DISTANCE_TO_SURFACE, M_PAIRING_DISTANCE_METRIC set to M_MANHATTAN and the M_DISTANCE_METRIC_OPERATOR set to M_SIGNED, M_SIGNED is ignored and M_ABSOLUTE is used instead. When more than one setting is available, and an unavailable setting is specified, an error is thrown. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	M_AUTO is the same as the first available setting: M_DISTANCE_TO_SURFACE, M_DISTANCE_TO_NEIGHBOR, M_DISTANCE_TO_REFERENCE in decreasing preference.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_NEIGHBOR	Specifies to calculate the distance between the points of the source point cloud or depth map, and their nearest point in the specified reference point cloud or depth map. Note, for this pairing type, only the following reference objects are supported and must be used with the following settings: Reference object. — M_PAIRING_DISTANCE_METRIC — M_DISTANCE_METRIC_OPERATOR Depth map. — M_DIRECTION_Z — M_ABSOLUTE or M_SIGNED. Point cloud container. — M_EUCLIDEAN or M_DIRECTION_NORMAL. — M_ABSOLUTE	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_REFERENCE	Specifies to calculate the distance between the points of the source point cloud or depth map and the specified reference 3D geometry. The paired reference point depends on the type of 3D geometry object specified: • Sphere: ○ Calculates the pairing distance to the center of the sphere. • Point: ○ Calculates the pairing distance to the point. • Cylinder: ○ Calculates the pairing distance to the nearest position on the central axis of the cylinder. The central axis is extended infinitely, even if the cylinder is finite. • Line: ○ Calculates the pairing distance to the nearest position on the line. Unlike the central axis of the cylinder, a finite line is not extended infinitely for this calculation. M_PAIRING_DISTANCE_METRIC must be set to M_EUCLIDEAN. M_DISTANCE_METRIC_OPERATOR can be set to M_ABSOLUTE or M_SQUARED.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISTANCE_TO_SURFACE	Specifies to calculate the distance between the points of the source point cloud or depth map, and the surface of the specified reference object. Note, for this pairing type, only the following reference objects are supported and must be used with the following settings: Reference object. | M_PAIRING_DISTANCE_METRIC | M_DISTANCE_METRIC_OPERATOR Sphere. | M_EUCLIDEAN | M_ABSOLUTE or M_SIGNED. Plane, box, or cylinder. | M_EUCLIDEAN | M_ABSOLUTE or M_SQUARED. Plane. | M_DIRECTION_Z | M_ABSOLUTE or M_SIGNED. Box. | M_MANHATTAN | M_ABSOLUTE Point cloud container. | M_EUCLIDEAN or M_DIRECTION_NORMAL. | M_ABSOLUTE	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_PAIRED_REFERENCE_FACET	Sets whether to save the index of the reference facet (face) paired to each source point. This control type is only available when M_PAIRING_TYPE is set to M_DISTANCE_TO_SURFACE, and the reference object is a geometry with multiple facets (for example, a box has 6 facets) or a meshed point cloud. If the reference object is a meshed point cloud, the index is the index of the paired mesh triangle of the M_COMPONENT_MESH_AIL component. If the reference object is a box, the surface index indicates the face of the box. The sides are indexed as follows:  If the reference object is a finite cylinder, index 0 is the start face, index 1 is the end face, and index 2 is the cylindrical face. Note, setting is only available if M_PAIRING_TYPE is set to M_DISTANCE_TO_SURFACE and the reference object is a box, finite cylinder, or point cloud; otherwise, an error is thrown. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not save the paired reference facet.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save the paired reference facet.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_PAIRED_REFERENCE_PIXEL	Sets whether to save the index of the paired reference pixel to each source point. Note, this setting is only available if M_PAIRING_TYPE is set to M_DISTANCE_TO_NEIGHBOR; otherwise, an error is thrown. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not save the paired reference pixel.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save the paired reference pixel.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAVE_PAIRED_REFERENCE_POINT		Sets whether to save the coordinates of the paired reference point to each source point. Note, this setting is only available if M_PAIRING_TYPE is set to M_DISTANCE_TO_SURFACE or M_DISTANCE_TO_NEIGHBOR, otherwise, an error is thrown. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not save the coordinates of the paired reference points.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save the coordinates of the paired reference points.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void M3dmetControlInt64 (AIL_ID ContextOrResult3dmetId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResult3dmetId

See ContextOrResult3dmetId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void M3dmetControlDouble (AIL_ID ContextOrResult3dmetId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResult3dmetId

See ContextOrResult3dmetId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetControlDraw

Synopsis

Control a setting of a draw 3D metrology context.

Syntax

```
void M3dmetControlDraw(
    AIL_ID DrawContext3dmetId, //out
    AIL_INT64 Operation,       //in
    AIL_INT64 ControlType,     //in
    AIL_DOUBLE ControlValue    //in
)
```

Description

This function controls a specified setting of a draw 3D metrology context. These settings establish which metrology results to draw into the 3D graphics list (and how to draw them), when calling M3dmetDraw3d().

You can inquire about most of these settings using M3dmetInquireDraw().

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **M3dmetControlDraw()** internally calls the *AIL_DOUBLE* version of this function (M3dmetControlDrawDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (M3dmetControlDrawInt64()).

Parameters

DrawContext3dmetId

Specifies the identifier of the draw 3D metrology context to control. The 3D metrology context must have been previously allocated on the system using M3dmetAlloc() with M_DRAW_3D_CONTEXT.

Operation

Specifies the draw operation.

See the Parameter associations section for possible values that can be specified.

ControlType

Specifies the setting to change.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **Operation**, **ControlType**, and **ControlValue** parameters.

- For controlling a draw 3D metrology context setting

The following Operation, ControlType, and ControlValue parameter settings are available to control draw 3D metrology context settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a draw 3D metrology context setting																		
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlType																
ControlValue																
M_ALL	Applies the setting to all operations that support the control type.															
M_ACTIVE	Sets whether to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies to use the default setting of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_APPEARANCE	Sets the appearance of the graphic(s) to solid surface, wireframe, or points. The color of the points, wireframe, or outline is determined by M_COLOR, while the color of the solid surface is determined by the default fill color of the 3D graphics list (M3dgraControl() with M_FILL_COLOR).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use the default setting of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINTS	Specifies a points appearance. The graphic(s) appear as a set of points representing its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID	Specifies a solid appearance.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_POINTS	Specifies a solid appearance with points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_WIREFRAME	Specifies a solid appearance with a wireframe.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIREFRAME	Specifies a wireframe appearance. The graphic(s) appear as a set of lines connecting its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR	Sets the color with which to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use the default setting of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the color.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															
	Green Specifies the green band, as a value between 0 and 255.															
	Blue Specifies the blue band, as a value between 0 and 255.															
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPACITY	Sets the opacity with which to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use the default setting of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the opacity of the 3D graphic. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKNESS	Sets the thickness with which to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use the default setting of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the point/line thickness, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_VOLUME_ELEMENTS	Sets whether and how to draw the volume elements used to compute the volume, independent of whether the elements contributed positively or negatively to the volume.	c		e	f	g	a		i		j	k	l	m	n	b

M_ACTIVE	Sets whether to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_APPEARANCE	Sets whether to draw the graphic(s) as a solid surface, wireframe, or points. The color of the points, wireframe, or outline is determined by M_COLOR, while the color of the solid surface is determined by the default fill color of the 3D graphics list (M3dgraControl() with M_FILL_COLOR). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SOLID. Same as M_SOLID.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINTS	Specifies a points appearance. The graphic(s) appear as a set of points representing its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID	Specifies a solid appearance.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_POINTS	Specifies a solid appearance with points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_WIREFRAME	Specifies a solid appearance with a wireframe.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIREFRAME	Specifies a wireframe appearance. The graphic(s) appear as a set of lines connecting its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR	Sets the color with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the color.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															
	Green Specifies the green band, as a value between 0 and 255.															
	Blue Specifies the blue band, as a value between 0 and 255.															
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPACITY		Sets the opacity with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 100.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the opacity of the 3D graphic. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKNESS		Sets the thickness with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1 pixel. Specifies the default value; the default value is 1 pixel.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the point/line thickness, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_VOLUME_ELEMENT_APPEARANCE	Sets whether to draw the volume element(s) as a surface or volume. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_VOLUME.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE	Specifies to draw the surfaces of the source and reference objects that were specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_REFERENCE	Specifies to draw only the surfaces of the reference object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_SOURCE	Specifies to draw only the surfaces of the source object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME	Specifies to draw each volume element as a distinguishable volume. The volume elements are the individual volumes that were used to calculate the volume during the M3dmetVolumeEx() operation. Depending on the source and reference objects specified for the volume operation, Aurora Imaging Library draws each volume element as a triangular prism (point cloud source with a plane reference), a triangular pyramid (point cloud source without a reference), or a rectangular prism (depth map source with or without a reference). > Note that to render transparency for a volume (that is, when M_OPACITY is less than 100.0), a powerful GPU is required, especially when the number of volume elements is large.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_VOLUME_NEGATIVE_ELEMENTS	Sets whether and how to draw the volume elements that contributed negatively to the volume computation; these elements decreased the volume.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACTIVE	Sets whether to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_APPEARANCE	Sets the appearance of the graphic(s) to solid surface, wireframe, or points. The color of the points, wireframe, or outline is determined by M_COLOR, while the color of the solid surface is determined by the default fill color of the 3D graphics list (M3dgraControl() with M_FILL_COLOR). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SOLID. Same as M_SOLID.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINTS	Specifies a points appearance. The graphic(s) appear as a set of points representing its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID	Specifies a solid appearance.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_POINTS	Specifies a solid appearance with points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_WIREFRAME	Specifies a solid appearance with a wireframe.	c		e	f	g	a		i		j	k	l	m	n	b

M_WIREFRAME	Specifies a wireframe appearance. The graphic(s) appear as a set of lines connecting its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR	Sets the color with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_RED.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the color.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															
	Green Specifies the green band, as a value between 0 and 255.															
	Blue Specifies the blue band, as a value between 0 and 255.															
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPACITY	Sets the opacity with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the opacity of the 3D graphic. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKNESS	Sets the thickness with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1 pixel. Specifies the default value; the default value is 1 pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the point/line thickness, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_ELEMENT_APPEARANCE	Sets whether to draw the volume element(s) as a surface or volume. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_VOLUME. Same as M_VOLUME.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE	Specifies to draw the surfaces of the source and reference objects that were specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_REFERENCE	Specifies to draw only the surfaces of the reference object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_SOURCE	Specifies to draw only the surfaces of the source object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME	Specifies to draw each volume element as a distinguishable volume. The volume elements are the individual volumes that were used to calculate the volume during the M3dmetVolumeEx() operation. Depending on the source and reference objects specified for the volume operation, Aurora Imaging Library draws each volume element as a triangular prism (point cloud source with a plane reference), a triangular pyramid (point cloud source without a reference), or a rectangular prism (depth map source with or without a reference). > Note that to render transparency for a volume (that is, when M_OPACITY is less than 100.0), a powerful GPU is required, especially when the number of volume elements is large.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_VOLUME_POSITIVE_ELEMENTS	Sets whether and how to draw the volume elements that contributed positively to the volume computation; these elements increased the volume.	c		e	f	g	a		i		j	k	l	m	n	b

M_ACTIVE	Sets whether to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_APPEARANCE	Sets the appearance of the graphic(s) to solid surface, wireframe, or points. The color of the points, wireframe, or outline is determined by M_COLOR, while the color of the solid surface is determined by the default fill color of the 3D graphics list (M3dgraControl() with M_FILL_COLOR). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SOLID. Same as M_SOLID.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINTS	Specifies a points appearance. The graphic(s) appear as a set of points representing its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID	Specifies a solid appearance.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_POINTS	Specifies a solid appearance with points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_WIREFRAME	Specifies a solid appearance with a wireframe.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIREFRAME	Specifies a wireframe appearance. The graphic(s) appear as a set of lines connecting its vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR	Sets the color with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COLOR_GREEN.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the color.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															
	Green Specifies the green band, as a value between 0 and 255.															
	Blue Specifies the blue band, as a value between 0 and 255.															
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPACITY	Sets the opacity with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the opacity of the 3D graphic. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKNESS	Sets the thickness with which to draw the graphic(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1 pixel. Specifies the default value; the default value is 1 pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the point/line thickness, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_VOLUME_ELEMENT_APPEARANCE	Sets whether to draw the volume element(s) as a surface or volume. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_VOLUME. Same as M_VOLUME.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE	Specifies to draw the surfaces of the source and reference objects that were specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_REFERENCE	Specifies to draw only the surfaces of the reference object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_SOURCE	Specifies to draw only the surfaces of the source object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME	Specifies to draw each volume element as a distinguishable volume. The volume elements are the individual volumes that were used to calculate the volume during the M3dmetVolumeEx() operation. Depending on the source and reference objects specified for the volume operation, Aurora Imaging Library draws each volume element as a triangular prism (point cloud source with a plane reference), a triangular pyramid (point cloud source without a reference), or a rectangular prism (depth map source with or without a reference). > Note that to render transparency for a volume (that is, when M_OPACITY is less than 100.0), a powerful GPU is required, especially when the number of volume elements is large.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_DRAW_SETTINGS	Specifies a global draw 3D metrology context setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_ELEMENT_INDEX	Sets the index of the volume element to draw. You can use M3dmetCopyResult() with M_VOLUME_ELEMENT_INDEX_IMAGE and M_VOLUME_ELEMENT_STATUS_IMAGE to determine which volume elements were used in the volume computation and whether the contribution was positive, negative, or both. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to draw all the volume elements.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void M3dmetControlDrawDouble (AIL_ID DrawContext3dmetId, AIL_INT64 Operation, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

DrawContext3dmetId

See DrawContext3dmetId of the main function for a description.

Operation

See Operation of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void M3dmetControlDrawInt64 (AIL_ID DrawContext3dmetId, AIL_INT64 Operation, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

DrawContext3dmetId

See DrawContext3dmetId of the main function for a description.

Operation

See Operation of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetCopy

Synopsis

Copy a 3D geometry into, or out of, a fit 3D metrology context.

Syntax

```
void M3dmetCopy(
    AIL_ID SrcAilObjectId, //in
    AIL_ID DstAilObjectId, //out
    AIL_INT64 CopyType,    //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function copies a 3D geometry from a 3D geometry object into a fit 3D metrology context, or vice versa. The fit operation (M3dmetFit()) will use this geometry to determine an initial fit estimate if M3dmetControl() with M_ESTIMATION_MODE is set to M_FROM_GEOMETRY. Note that the fit operation does not support 3D box geometries.

Parameters

SrcAilObjectId

Specifies the Aurora Imaging Library object from which to copy the 3D geometry.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source Aurora Imaging Library object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies to copy the XY (Z=0) plane. This requires that the destination Aurora Imaging Library object be a fit 3D metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of the 3D geometry object from which to copy the 3D geometry; 3D box geometries are not supported.	c		e	f	g	a		i		j	k	l	m	n	b
Fit 3D metrology context identifier	Specifies the identifier of the fit 3D metrology context from which to copy the 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b

DstAilObjectId

Specifies the identifier of the Aurora Imaging Library object in which to copy the 3D geometry.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination Aurora Imaging Library object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
3D geometry object identifier	Specifies the identifier of the 3D geometry object in which to copy the 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
Fit 3D metrology context identifier	Specifies the identifier of the fit 3D metrology context in which to copy the 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b

CopyType

Specifies the type of copy operation to perform.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Specifies the type of copy operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_ESTIMATE_GEOMETRY	Specifies to copy the 3D geometry from the source Aurora Imaging Library object into the destination Aurora Imaging Library object. The 3D geometry in the destination Aurora Imaging Library object will be overwritten.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetCopyResult

Synopsis

Copy a group of results from a 3D metrology result buffer into an image buffer, a 3D geometry object, a point cloud container, or a transformation matrix object.

Syntax

```
void M3dmetCopyResult(
    AIL_ID SrcResult3dmetId, //in
    AIL_ID DstAilObjectId,   //out
    AIL_INT64 CopyType,      //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function copies a group of results from a 3D metrology result buffer into an image buffer, a 3D geometry object, a point cloud container, or a transformation matrix object.

When copying results of an M3dmetFeatureEx() operation, there must be at least one calculated 3D geometry or matrix in the specified calculate 3D metrology result buffer. To retrieve the number of geometries or matrices in the result buffer, use M3dmetGetResult() with M_NUMBER.

When copying results of an M3dmetDistanceEx(), M3dmetFit(), or M3dmetVolumeEx() operation, the copy operations that are available depend on the resulting status of the distance, fit, or volume operation. To retrieve the status, use M3dmetGetResult() with M_STATUS_DISTANCE, M_STATUS_FIT, or M_STATUS_VOLUME, respectively.

Parameters

SrcResult3dmetId

Specifies the identifier of a 3D metrology result buffer from which to copy results. The 3D metrology result buffer must have been allocated using M3dmetAllocResult(); the result buffer must contain the results of a call to M3dmetDistanceEx(), M3dmetFeatureEx(), M3dmetFit(), or M3dmetVolumeEx().

See the Parameter associations section for possible values that can be specified.

DstAilObjectId

Specifies the identifier of the image buffer, 3D geometry object, point cloud container, or transformation matrix object into which the results will be copied.

See the Parameter associations section for possible values that can be returned.

CopyType

Specifies the type of copy operation to perform.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **SrcResult3dmetId** and **CopyType** parameters and possible values returned to the **DstAilObjectId** parameter.

- For specifying the copy type and destination object for a 3D metrology result buffer

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the copy type and destination object for a 3D metrology result buffer																		
SrcResult3dmetId			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	CopyType																	
		DstAilObjectId - Possible values returned																

Calculate 3D metrology result buffer ID, for copying results of an M3dmetDistanceEx operation	Specifies the identifier of a calculate 3D metrology result buffer from which to copy results of an M3dmetDistanceEx() operation. The result buffer must have been previously allocated using M3dmetAllocResult() with M_CALCULATE_RESULT.														
M_DISTANCE_NORMAL_IMAGE	Specifies to copy an image of the normal distance between the source and the reference points into an image buffer. For each source point, the operation copies the distance to the corresponding pixel location in the image buffer. Note that when dealing with a source point cloud or depth map container, the distance is set in the image buffer at the same location as the point's location in the range component of its container. Points that are not paired with a reference point are given the value of -1. To use this copy type, M3dmetControl() with M_DISTANCE_NORMAL must have been enabled prior to calling M3dmetDistanceEx() and M3dmetDistanceEx() must be completed.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 32-bit float buffer. The image buffer size must equal the size of the source container (range component) or source depth map image buffer used for the distance calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_POINT_IMAGE	Specifies to copy an image of the distance between the source and the reference points into an image buffer. For each source point, the operation copies the distance to the corresponding pixel location in the image buffer. Note that when dealing with a source point cloud or depth map container, the distance is set in the image buffer at the same location as the point's location in the range component of its container. Points that are not paired with a reference point are given the value of -1. To use this copy type, M3dmetDistanceEx() must be completed.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 32-bit float buffer. The image buffer size must equal the size of the source container (range component) or source depth map image buffer used for the distance calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRED_REFERENCE_FACET_IMAGE	Specifies to copy an image of the paired reference facets to the source points. To use this copy type, M3dmetControl() with M_SAVE_PAIRED_REFERENCE_FACET must have been enabled prior to calling M3dmetDistanceEx() and M3dmetDistanceEx() must be completed.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 32-bit unsigned buffer. The image buffer size must equal the size of the source container (range component) or source depth map image buffer used for the distance calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRED_REFERENCE_PIXEL_X_IMAGE	Specifies to copy an image of the paired reference pixel in the X-direction between the source and the reference points. To use this copy type, M3dmetControl() with M_SAVE_PAIRED_REFERENCE_PIXEL must have been enabled prior to calling M3dmetDistanceEx() and M3dmetDistanceEx() must be completed.	c		e	f	g	a		i		j	k	l	m	n	b

Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 32-bit unsigned buffer. The image buffer size must equal the size of the source container (range component) or source depth map image buffer used for the distance calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRED_REFERENCE_PIXEL_Y_IMAGE	Specifies to copy an image of the paired reference pixel coordinate in the Y-direction between the source and reference points. To use this copy type, M3dmetControl() with M_SAVE_PAIRED_REFERENCE_PIXEL must have been enabled prior to calling M3dmetDistanceEx() and M3dmetDistanceEx() must be completed.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 32-bit unsigned buffer. The image buffer size must equal the size of the source container (range component) or source depth map image buffer used for the distance calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_POINT_IMAGE	Specifies to copy an image of the paired reference points' coordinates used in the distance calculation. To use this copy type, M3dmetControl() with M_SAVE_PAIRED_REFERENCE_POINT must have been enabled prior to calling M3dmetDistanceEx() and M3dmetDistanceEx() must be completed.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 3-band, 32-bit float buffer. The image buffer size must equal the size of the source container (range component) or source depth map image buffer used for the distance calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
Calculate 3D metrology result buffer ID, for copying results of an M3dmetFeatureEx operation	Specifies the identifier of a calculate 3D metrology result buffer from which to copy results of an M3dmetFeatureEx() operation. The result buffer must have been previously allocated using M3dmetAllocResult() with M_CALCULATE_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATED_OBJECT(AIL_INT N)	Specifies to copy the Nth calculated object. Note that you can use M3dmetGetResult() with M_CALCULATED_OBJECT_TYPE to retrieve the type of object stored in the calculate 3D metrology result buffer. When you know the object type, you can allocate the appropriate destination object to receive the copied results, either a 3D geometry object or transformation matrix object.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	N Specifies the index of the geometry or matrix.														
	0 <= *Value* < M_NUMBER Specifies the valid index of the geometry or matrix.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object in which to copy results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b

3D transformation matrix object identifier	Specifies the identifier of a 3D transformation matrix object in which to copy results. The 3D transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
Calculate 3D metrology result buffer ID, for copying results of an M3dmetVolumeEx operation	Specifies the identifier of a calculate 3D metrology result buffer from which to copy results of an M3dmetVolumeEx() operation. The result buffer must have been previously allocated using M3dmetAllocResult() with M_CALCULATE_RESULT. Some copy types require the volume operation to have completed with a specific status.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_ELEMENT_INDEX_IMAGE	Specifies to create an image in which a pixel's gray value is set to the index of the volume element that was used to compute the volume. For unused volume elements, corresponding destination pixels are set to the maximum buffer value. A volume element's index is its numbered position in the source container's mesh component or in the source depth map image buffer. For a container, the mesh component is one-dimensional, and indexing begins at 0. For a depth map, indexing begins at 0 for the upper left position and proceeds from left to right across all rows in the 2D pixel grid. You can use the index image to determine which volume elements were used in the volume computation. To draw a single volume element, you can set its index in a draw 3D metrology context (using M3dmetControlDraw() with M_VOLUME_ELEMENT_INDEX), and then draw the element using M3dmetDraw3d(). To use this copy type, M3dmetControl() with M_SAVE_VOLUME_INFO must have been enabled prior to calling M3dmetVolumeEx().	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 32-bit unsigned buffer. When the source is a container, the image buffer must be one-dimensional with a size in X equal to the mesh component's size in X. When the source is a depth map, the image buffer's size must match the depth map's size, in both X and Y. To retrieve the size of the source container (mesh component) or depth map, use M3dmetGetResult() with M_RESULT_ELEMENT_IMAGE_SIZE_X and M_RESULT_ELEMENT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_ELEMENT_MASK	Specifies to create a mask image whose pixels are non-zero for volume elements that were used for the volume computation, and zero otherwise. This operation assigns a non-zero value to the volume element's corresponding pixel location in the destination image buffer. The non-zero value is 1 for floating-point image buffers and -1 for signed image buffers. For unsigned image buffers, the maximum buffer value is used. All other pixels in the buffer are set to 0. To use this copy type, M3dmetControl() with M_SAVE_VOLUME_INFO must have been enabled prior to calling M3dmetVolumeEx().	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band buffer. When the source is a container, the image buffer must be one-dimensional with a size in X equal to the mesh component's size in X. When the source is a depth map, the image buffer's size must match the depth map's size, in both X and Y. To retrieve the size of the source container (mesh component) or depth map, use M3dmetGetResult() with M_RESULT_ELEMENT_IMAGE_SIZE_X and M_RESULT_ELEMENT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b

			c		e	f	g	a		i		j	k	l	m	n	b
	M_VOLUME_ELEMENT_STATUS_IMAGE	Specifies to create a status image whose pixels represent the status of source volume elements. A volume element's status indicates its contribution to the volume calculation, either positive, negative, a mixture of both, or not at all. Each status value is assigned to a destination image pixel that corresponds to the volume element's index position in the source container's mesh component (or in the source depth map image buffer). For a container, the mesh component is one-dimensional, and indexing begins at 0. For a depth map, indexing begins at 0 for the upper left position and proceeds from left to right across all rows in the 2D pixel grid. Aurora Imaging Library assigns the following values to destination image pixels to indicate the source volume element status. Note that spliced volume elements are those that intersect the reference object (only for source point clouds). Unused | Positive | Negative | Both 0 | 1 (0b00000001) | 2 (0b00000010) | 3 (0b00000011) Unused Spliced | Positive Spliced | Negative Spliced | Both Spliced 4 (0b00000100) | 5 (0b00000101) | 6 (0b00000110) | 7 (0b00000111) For a source depth map, status values are limited to unused, positive, or negative (0, 1, or 2). For a source point cloud, any of the eight status values are possible. Note that for depth maps, this copy type and M_VOLUME_SOURCE_POINTS_STATUS_IMAGE create identical status images. You can map the status image through a LUT to display the volume elements' status in different colors, which can help you visualize each volume element's contribution to the volume operation, for diagnostic purposes. Typically, M_VOLUME_ELEMENT_STATUS_IMAGE generates a status image that is more useful than M_VOLUME_SOURCE_POINTS_STATUS_IMAGE, since volume elements truly indicate the individual volumes used for the volume computation. To use this copy type, M3dmetControl() with M_SAVE_VOLUME_INFO must have been enabled prior to calling M3dmetVolumeEx().	c		e	f	g	a		i		j	k	l	m	n	b
	Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band buffer. When the source is a container, the image buffer must be one-dimensional with a size in X equal to the mesh component's size in X. When the source is a depth map, the image buffer's size must match the depth map's size, in both X and Y. To retrieve the size of the source container (mesh component) or depth map, use M3dmetGetResult() with M_RESULT_ELEMENT_IMAGE_SIZE_X and M_RESULT_ELEMENT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b

M_VOLUME_REFERENCE_CONTAINER	Specifies to copy projected source points into a container. The destination container will consist of coplanar points coincident upon the reference plane. These points result from a source point projection on the reference plane to indicate which points and triangles on the reference plane were used in the volume calculation. Note that the reference (destination) container will hold projected points from all source points, regardless of the volume mode setting. The destination container's size equals that of the source container's range component, in X and Y. The destination container's mesh component will have an X-size that equals the number of mesh triangles used to compute the volume. Note that a mesh component is automatically generated for the destination container. You can use this container to help create a 3D drawing of volume elements. However, the index, mask, or status images that you can create using this function (for example, M_VOLUME_ELEMENT_MASK) are typically more useful as diagnostic tools. To use this copy type, M3dmetControl() with M_SAVE_VOLUME_INFO must have been enabled prior to calling M3dmetVolumeEx(). This copy type is available only when a plane geometry or M_XY_PLANE was used as the reference object for the volume computation.	c		e	f	g	a		i		j	k	l	m	n	b
Container identifier	Specifies the destination container identifier. The container must have been previously allocated using MbufAllocContainer() with M_PROC, and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_REFERENCE_DEPTH_MAP	Specifies to copy volume element depths into a depth map image buffer. If the reference object was a plane, the resulting depth map's pixel values will represent the depth from the boundary of the depth map to the reference plane, for the corresponding source pixel. If the reference object was a depth map, the resulting depth map will be a copy of the reference depth map, with the following exception: any invalid pixel in the source depth map will also invalidate the corresponding pixel in the resulting depth map image buffer. To use this copy type, M3dmetControl() with M_SAVE_VOLUME_INFO must have been enabled prior to calling M3dmetVolumeEx(). This copy type is available only when the source object for the volume computation was a depth map.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band buffer. The image buffer size must equal the size of the depth map used for the volume calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_SOURCE_POINTS_MASK	Specifies to create a mask image whose pixels are non-zero for source points that contributed to the volume computation and zero otherwise. When the source is a point cloud, these are the points that define the triangles of the point cloud's mesh, and for which contributing volume elements were calculated. This operation uses the source point's location in its container or depth map to assign a non-zero value to a corresponding pixel location in the image buffer. The non-zero value is 1 for floating-point image buffers and -1 for signed image buffers. For unsigned image buffers, the maximum buffer value is used. All other pixels in the buffer are set to 0. You can use the mask image to crop out points that were not used to compute the volume. To do so, use M3dimCrop(). To use this copy type, M3dmetControl() with M_SAVE_VOLUME_INFO must have been enabled prior to calling M3dmetVolumeEx().	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band buffer. The image buffer size must equal the size of the container (range component) or depth map used for the volume calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b

M_VOLUME_SOURCE_POINTS_STATUS_IMAGE	Specifies to create a status image whose pixels represent the status of source points. A point's status indicates its contribution to the volume calculation, either positive, negative, a mixture of both, or not at all. This operation uses the source point's location in its container or depth map to assign a status value to a corresponding pixel location in the image buffer. Aurora Imaging Library assigns the following values to image pixels to indicate the point status. Unused: 0 Positive: 1 (0b00000001) Negative: 2 (0b00000010) Both: 3 (0b00000011) For a source depth map, status values are limited to unused, positive, or negative (0, 1, or 2). Note that for depth maps, this copy type and M_VOLUME_ELEMENT_STATUS_IMAGE create identical status images. You can map the status image through a LUT to display the source points' status in different colors, which can help you visualize each point's contribution to the volume operation, for diagnostic purposes. If you want to crop points based on their status, use M3dimCrop(). Note that you must appropriately process the status image before passing it to M3dimCrop(). For example, to ensure that pixels indicating a positive status are kept while the rest are set to 0, use MimArith() with M_AND_CONST to perform a bitwise AND operation; then, pass the resulting image to M3dimCrop(), which will reject points that correspond to zero-valued pixels in the image. To use this copy type, M3dmetControl() with M_SAVE_VOLUME_INFO must have been enabled prior to calling M3dmetVolumeEx().	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band buffer. The image buffer size must equal the size of the container (range component) or depth map used for the volume calculation, in both X and Y. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
Fit 3D metrology result buffer ID from which to copy	Specifies the identifier of a fit 3D metrology result buffer from which to copy results of an M3dmetFit() operation. The result buffer must have been previously allocated using M3dmetAllocResult() with M_FIT_RESULT. Some copy types require the fit operation to have completed with a specific status.	c		e	f	g	a		i		j	k	l	m	n	b
M_FITTED_GEOMETRY	Specifies to copy the fitted 3D geometry object; if you specify this copy type, the copied 3D cylinder geometry object or 3D line geometry object will always be finite. Note that you can perform this copy operation only if the fit completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object in which to copy results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
M_FITTED_GEOMETRY_INFINITE	Specifies to copy the fitted 3D geometry object; if you specify this copy type, the copied 3D cylinder geometry object or 3D line geometry object will always be infinite. Note that you can perform this copy operation only if the fit completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object in which to copy results. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIXTURING_MATRIX	Specifies to copy, into the transformation matrix, the transformation that can move the working coordinate system to the fitted 3D geometry. If the fitted geometry is a plane, the transformation moves the working coordinate system's X- and Y-axis to the fitted plane, with the origin at the centroid of all inlier points. The plane's normal becomes the Z-axis. If the fitted geometry is a cylinder or line, the transformation moves the working coordinate system such that its origin is at the starting point of the geometry, with the Z-axis aligned to the central axis. If the fitted geometry is a sphere, the transformation moves the working coordinate system such that its origin is at the center of the sphere. Note that you can only perform this copy operation if the fit operation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
3D transformation matrix object identifier	Specifies the identifier of a 3D transformation matrix object in which to copy results. The 3D transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_INLIER_MASK	Specifies to copy the mask constructed by the points that were considered inliers during the fit. That is, points that were considered inliers will have a confidence score of 255 in the destination image buffer, and all other points will have a confidence score of 0. Information about a point is set in the image buffer at the same location as the point's location in the range component of its point cloud container. Note that you can perform this copy operation whether the fit operation completed successfully or not, as long as a call to M3dmetFit() was made.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 8-bit unsigned image buffer. The image buffer size must have the same size as the container or depth map used during the fit operation. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_MASK	Specifies to copy the mask constructed by the points that were considered outliers during the fit. That is, points that were considered outliers will have a confidence score of 255 in the destination image buffer, and all other points will have a confidence score of 0. Information about a point is set in the image buffer at the same location as the point's location in the range component of its point cloud container. Note that, you can perform this copy operation whether the fit operation completed successfully or not, as long as a call to M3dmetFit() was made.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer in which to copy results. The image buffer must be a 1-band, 8-bit unsigned image buffer. The image buffer size must have the same size of the container or depth map used during the fit operation. To retrieve the size, use M3dmetGetResult() with M_RESULT_IMAGE_SIZE_X and M_RESULT_IMAGE_SIZE_Y. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.

DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetDistance

Synopsis

Calculates the distance between a point cloud or depth map, and a point cloud, depth map, or 3D geometry object.

Syntax

```
void M3dmetDistance(
    AIL_ID SrcContainerOrImageBufId, //in
    AIL_ID RefAilObjectId,           //in
    AIL_ID DstImageBufId,            //out
    AIL_INT64 DistanceType,          //in
    AIL_DOUBLE Param,                //in
    AIL_INT64 ControlFlag            //in
)
```

Description

This function calculates various distance measurements between a point cloud or depth map, and a point cloud, depth map, or 3D geometry.

This function is similar to M3dmetStat(), but does not require allocating any 3D metrology contexts or result buffers, and does not calculate any statistical metrics based on the distance measurements.

Parameters

SrcContainerOrImageBufId

Specifies the source point cloud container, depth map container, or depth map image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud or depth map																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Depth map container identifier	Specifies the identifier of a depth map container. The container must store data in a 3D-processable depth map format. You can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map. The container must not have a region component.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer must not have a region of interest (ROI) associated with it.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies the identifier of a point cloud container. The container must be 3D-processable. You can use MbufInquireContainer() with M_3D_PROCESSABLE to ensure that the container contains a 3D-processable point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

RefAilObjectId

Specifies the reference object with respect to which distances will be measured.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the reference Aurora Imaging Library object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane. > This is equivalent to passing a 3D plane geometry object, and can therefore be used with the same DistanceType parameter settings.	c		e	f	g	a		i		j	k	l	m	n	b

3D geometry object identifier	Specifies the identifier of a reference 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include box, cylinder, line, plane, point, and sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map container identifier	Specifies the identifier of a depth map container. The container must store data in a 3D-processable depth map format. You can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map. The container must not have a region component.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer is allocated using MbufAlloc2d(), and must not have a region of interest (ROI) associated with it.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies the identifier of a point cloud container. The container must be 3D-processable. You can use MbufInquireContainer() with M_3D_PROCESSABLE to ensure that the container contains a 3D-processable point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

See the Parameter associations section for possible values that can be specified.

DstImageBufId

Specifies the identifier of the buffer in which to store the calculated distances.

The buffer must be a 1-band, 32-bit floating-point image buffer. The destination image buffer must be the same size as the source buffer (or, if the source is a container, the same size as the range component). You can inquire the size of the source object using MbufInquire() (or, for a container, MbufInquireContainer() with M_COMPONENT_RANGE) with M_SIZE_X and M_SIZE_Y. These values can then be used to allocate an image buffer of the correct size.

The image buffer must not have a region of interest (ROI) associated with it.

> Note that, if the source object is an image buffer, its X and Y (but not Z) calibration information will be copied into the destination image buffer. Otherwise, the destination image buffer will be uncalibrated.

DistanceType

Specifies the type of distance to calculate.

See the Parameter associations section for possible values that can be specified.

Param

Specifies an optional maximum distance for which to complete calculations when the reference Aurora Imaging Library object is a point cloud container. Specify a value if DistanceType is set to M_DISTANCE_TO_MESH or M_DISTANCE_TO_NEAREST_NEIGHBOR; otherwise, set this parameter to M_DEFAULT. Points determined to be further from the reference than this value are assigned a distance of **AIL_FLOAT_MAX** in the destination, instead of having their exact distances calculated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the maximum distance to calculate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that there is no maximum distance for which to complete calculations. All distance measurements will be fully calculated.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies an optional maximum distance. All distance measurements greater than this limit are returned as **AIL_FLOAT_MAX**.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **RefAilObjectId** and **DistanceType** parameters.

- For specifying the type of distance measurement for the specified reference object

To specify the type of distance measurement, the DistanceType parameter can be set to one of the following values, depending on the type of reference object (RefAilObjectId) being measured against.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of distance measurement for the specified reference object																		
RefAilObjectId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
DistanceType																
3D box geometry object identfier	Specifies a 3D box geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.															
M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance to the surface of the box. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_MANHATTAN_DISTANCE_TO_SURFACE	Specifies to calculate the Manhattan distance to the surface of the box. Distance measurements are positive for points outside the box, and negative for points inside the box.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance to the surface of the box. Distance measurements are positive for points outside the box, and negative for points inside the box.	c		e	f	g	a		i		j	k	l	m	n	b
3D cylinder geometry object identifier	Specifies a 3D cylinder geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance to the surface of the cylinder. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_CENTER_AXIS	Specifies to calculate the absolute distance to the cylinder's central axis. The axis extends infinitely in both directions, even if the cylinder has a finite size.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_CENTER_AXIS_SQUARED	Specifies to calculate the square of the distance to the cylinder's central axis. The axis extends infinitely in both directions, even if the cylinder has a finite size.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance to the surface of the cylinder. Distance measurements are positive for points outside the cylinder, and negative for points inside the cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies a 3D line geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_LINE	Specifies to calculate the distance to the line. The length of the line is respected; if the line is finite, it is not extended infinitely when calculating distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_LINE_SQUARED	Specifies to calculate the square of the distance to the line. The length of the line is respected; if the line is finite, it is not extended infinitely when calculating distances.	c		e	f	g	a		i		j	k	l	m	n	b

3D plane geometry object identifier	Specifies a 3D plane geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. When the reference object is a 3D plane geometry object, calculations are quicker if the source object is a depth map rather than a point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance, perpendicular to the plane. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_Z_TO_SURFACE	Specifies to calculate the absolute distance along the Z-axis to the plane. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance, perpendicular to the plane. Distance measurements are positive for points on the same side as the plane's normal, and negative on the other side.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_Z_TO_SURFACE	Specifies to calculate the signed distance along the Z-axis to the plane. Distance measurements are positive if the point is above the plane and negative below.	c		e	f	g	a		i		j	k	l	m	n	b
3D point geometry object identifier	Specifies a 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_POINT	Specifies to calculate the distance to the point.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_POINT_SQUARED	Specifies to calculate the square of the distance to the point.	c		e	f	g	a		i		j	k	l	m	n	b
3D sphere geometry object identifier	Specifies a 3D sphere geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a sphere.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance to the surface of the sphere. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_CENTER	Specifies to calculate the signed distance to the center of the sphere.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_CENTER_SQUARED	Specifies to calculate the square of the distance to the center of the sphere.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance to the surface of the sphere. Distance measurements are positive for points outside the sphere, and negative for points inside the sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map container identifier	Specifies a depth map container, allocated with MbufAllocContainer() with M_PROC. The container must not have a region component. When the source and reference Aurora Imaging Library objects are depth maps, calculations are completed more quickly.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_Z_TO_SURFACE	Specifies to calculate the absolute distance to the position in the depth map that is directly above or below the source along the Z-axis. The returned value is always positive. If a point would be projected outside of the depth map, **AIL_FLOAT_MAX** is returned.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SIGNED_DISTANCE_Z_TO_SURFACE	Specifies to calculate the signed distance using the Z-coordinate of the corresponding pixel of the reference depth map. That is, the distance is calculated along the Z-axis. Distance measurements are positive for points above the depth map, and negative for points below the depth map. If a point would be projected outside of the depth map, **AIL_FLOAT_MAX** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier		Specifies the identifier of a depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer must not have a region of interest (ROI) associated with it. When the source and reference Aurora Imaging Library objects are depth maps, calculations are completed more quickly. The speed is further increased if both depth maps use the same calibration context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE_DISTANCE_Z_TO_SURFACE	Specifies to calculate the absolute distance to the position in the depth map that is directly above or below the source along the Z-axis. The returned value is always positive. If a point would be projected outside of the depth map, **AIL_FLOAT_MAX** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED_DISTANCE_Z_TO_SURFACE	Specifies to calculate the signed distance using the Z-coordinate of the corresponding pixel of the reference depth map. That is, the distance is calculated along the Z-axis. Distance measurements are positive for points above the depth map, and negative for points below the depth map. If a point would be projected outside of the depth map, **AIL_FLOAT_MAX** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier		Specifies a point cloud container, allocated with MbufAllocContainer() with M_PROC. The function runs much slower when the reference Aurora Imaging Library object is a point cloud container. If speed is an issue, you can specify a maximum for calculating distance measurements using the Param parameter. Any distance measurements greater than the maximum value are returned as **AIL_FLOAT_MAX**. You can also project the reference point cloud into a fully corrected depth map first (using M3dimProject()), at the cost of precision.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_MESH	Specifies to calculate the shortest distance to the reference object's mesh. This calculation requires that the reference object have a mesh component.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_NEAREST_NEIGHBOR	Specifies to calculate the distance to the nearest point in the reference point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetDistanceEx

Synopsis

Calculates the distance between a point cloud or depth map, and a point cloud, depth map, or 3D geometry object, and allows you to store results in a result buffer.

Syntax

```
void M3dmetDistanceEx(
    AIL_ID DistanceContext3dmetId,       //in
    AIL_ID SrcContainerOrImageBufId,     //in
    AIL_ID RefAilObjectId,               //in
    AIL_ID Result3dmetIdOrDstImageBufId, //out
    AIL_INT64 ControlFlag                //in
)
```

Description

This function calculates various distance measurements between a point cloud or depth map, and a point cloud, depth map, or 3D geometry, and allows you to store results in a result buffer.

Parameters

DistanceContext3dmetId

Specifies the distance 3D metrology context for the distance computation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the distance 3D metrology context identifier

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default distance 3D metrology context of the current Aurora Imaging Library application. > The distance operation will use default values for all distance control types listed in M3dmetControl().	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_DIST_TO_NEIGHBOR	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_PAIRING_TYPE which is set to M_DISTANCE_TO_NEIGHBOR. Use this predefined context to calculate the distance between the points of the point cloud or depth map, and their nearest-neighbor points of the reference point cloud or depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_DIST_TO_NEIGHBOR_IN_DIRECTION_NORMAL	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_PAIRING_TYPE which is set to M_DISTANCE_TO_NEIGHBOR, and M_PAIRING_DISTANCE_METRIC which is set to M_DIRECTION_NORMAL. Use this predefined context to calculate the distance between the points of the source point cloud or depth map, and their nearest-neighbor point, in the direction of the source point's normal, in the reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_DIST_TO_REF	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_PAIRING_TYPE which is set to M_DISTANCE_TO_REFERENCE. Use this predefined context to calculate the distance between the points of the point cloud or depth map, and the reference object.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISTANCE_CONTEXT_IN_DIRECTION_NORMAL	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_PAIRING_DISTANCE_METRIC which is set to M_DIRECTION_NORMAL. Use this predefined context to calculate the distance between the points of the source point cloud, and the surface of the reference object in the normal direction of the source points.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_IN_DIRECTION_Z	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_PAIRING_DISTANCE_METRIC which is set to M_DIRECTION_Z. Use this predefined context to calculate the distance between the points of the point cloud or depth map, and their closest points the reference object in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_MANHATTAN_DIST	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_PAIRING_DISTANCE_METRIC which is set to M_MANHATTAN. Use this predefined context to calculate the distance between the points of the point cloud or depth map, and the points of the reference object using the Manhattan distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_SIGNED	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_DISTANCE_METRIC_OPERATOR which is set to M_SIGNED. Use this predefined context to calculate the signed distance between the points of the point cloud or depth map, and their closest point in the reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_SIGNED_DIST_IN_DIRECTION_NORMAL	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_DISTANCE_METRIC_OPERATOR which is set to M_SIGNED, and M_PAIRING_DISTANCE_METRIC which is set to M_DIRECTION_NORMAL. Use this predefined context to calculate the signed distance between the points of the point cloud, and their closest point in the reference object in the normal direction of the source points.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_SIGNED_DIST_IN_DIRECTION_Z	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_DISTANCE_METRIC_OPERATOR which is set to M_SIGNED, and M_PAIRING_DISTANCE_METRIC which is set to M_DIRECTION_Z. Use this predefined context to calculate the signed distance between the points of the point cloud or depth map, and their closest point in the reference object in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_CONTEXT_SQUARED	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_DISTANCE_METRIC_OPERATOR which is set to M_SQUARED. Use this predefined context to calculate the squared distance between the points of the point cloud or depth map, and their closest point in the reference object.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISTANCE_CONTEXT_SQUARED_DIST_TO_REF	Specifies a predefined distance 3D metrology context with all distance context control types (M3dmetControl()) set to their default, except M_DISTANCE_METRIC_OPERATOR which is set to M_SQUARED, and M_PAIRING_TYPE which is set to M_DISTANCE_TO_REFERENCE. Use this predefined context to calculate the squared distance between the points of the point cloud or depth map, and their closest point in the reference object.	c		e	f	g	a		i		j	k	l	m	n	b
Distance 3D metrology context identifier	Specifies the identifier of a distance 3D metrology context, previously allocated using M3dmetAlloc() with M_DISTANCE_CONTEXT. > If a previously allocated context is specified, the function applies the distance control settings specified using M3dmetControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerOrImageBufId

Specifies the source point cloud container, depth map container, or depth map image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud or depth map																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Depth map container identifier	Specifies the identifier of a depth map container. The container must store data in a 3D-processable depth map format. You can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map. The container must not have a region component.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer must not have a region of interest (ROI) associated with it.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies the identifier of a point cloud container. The container must be 3D-processable. You can use MbufInquireContainer() with M_3D_PROCESSABLE to ensure that the container contains a 3D-processable point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

RefAilObjectId

Specifies the reference object with respect to which distances will be measured. The reference object specified depends on the pairing type specified using M3dmetControl() with M_PAIRING_TYPE.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the reference Aurora Imaging Library object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane. The XY-plane can be used as the reference object when the pairing type corresponds to M_DISTANCE_TO_REFERENCE or M_DISTANCE_TO_SURFACE.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a reference 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined. Supported 3D geometries include box, cylinder, line, plane, point, and sphere. A 3D geometry object can be used as the reference object when the pairing type corresponds to M_DISTANCE_TO_REFERENCE or M_DISTANCE_TO_SURFACE. In the latter case, the 3D geometry cannot be a point geometry.	c		e	f	g	a		i		j	k	l	m	n	b

Depth map container identifier	Specifies the identifier of a depth map container. A depth map container can be used as the reference object when the pairing type corresponds to M_DISTANCE_TO_NEIGHBOR. The container must store data in a 3D-processable depth map format. You can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map. The container must not have a region component.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. A depth map image buffer can be used as the reference object when the pairing type corresponds to M_DISTANCE_TO_NEIGHBOR. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer is allocated using MbufAlloc2d(), and must not have a region of interest (ROI) associated with it.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies the identifier of a point cloud container. A point cloud container can be used as the reference object when the pairing type corresponds to M_DISTANCE_TO_NEIGHBOR or M_DISTANCE_TO_SURFACE. In the latter case, the 3D geometry cannot be a point geometry. The container must be 3D-processable. You can use MbufInquireContainer() with M_3D_PROCESSABLE to ensure that the container contains a 3D-processable point cloud. The container must also have at least a range and a confidence component.	c		e	f	g	a		i		j	k	l	m	n	b

Result3dmetIdOrDstImageBufId

Specifies the identifier of the container, image buffer, or a calculate 3D metrology result buffer in which to store the calculated distances.

The buffer must be a 1-band, 32-bit floating-point image buffer. The destination image buffer must be the same size as the source buffer (or, if the source is a container, the same size as the range component). You can inquire the size of the source object using MbufInquire() (or, for a container, MbufInquireContainer() with M_COMPONENT_RANGE) with M_SIZE_X and M_SIZE_Y. These values can then be used to allocate an image buffer of the correct size.

The image buffer must not have a region of interest (ROI) associated with it.

> Note that, if the source object is a depth map, its X and Y (but not Z) calibration information will be copied into the destination image buffer. Otherwise, the destination image buffer will be uncalibrated.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetDraw3d

Synopsis

Draw the result of a 3D metrology operation into a 3D graphics list.

Syntax

```
AIL_INT64 M3dmetDraw3d(
    AIL_ID OperationDraw3dContext3dmetId, //in
    AIL_ID SrcResult3dmetId,              //in
    AIL_ID DstList3dgraId,                //out
    AIL_INT64 DstParentLabel,             //in
    AIL_INT64 ControlFlag                 //in
)
```

Description

This function draws the specified result of a fit or volume 3D metrology operation, stored in the specified 3D metrology result buffer, into a 3D graphics list.

For fit results, call this function directly to perform the draw. No allocated context or specified control settings are required. Fitted 3D geometries are drawn using the default settings of the 3D graphics list. For volume results, you typically allocate a draw 3D metrology context using M3dmetAlloc() with M_DRAW_3D_CONTEXT. You then set draw operations and options for the draw using M3dmetControlDraw() before calling **M3dmetDraw3d**().

Prior to drawing results, the fit or volume operation must have completed successfully (that is, M3dmetGetResult() with M_STATUS_FIT or M_STATUS_VOLUME returns M_SUCCESS, respectively). For volume results, you must also have enabled M_SAVE_VOLUME_INFO (using M3dmetControl()) prior to performing the volume operation.

For fit results, this function can only draw the fitted 3D geometry. When drawing a fitted plane, this function will draw a 3D plane geometry clipped to the limits of the points that were used to fit the plane, unlike M3dgeoDraw3d(), which clips the 3D graphic using the 3D graphics list's clipping box.

For volume results, this function draws a meshed point cloud for diagnosing the volume computation. Use M3dmetControlDraw() to specify which volume elements to draw and how to draw them. A volume element is a triangular prism or pyramid that represents the computed 3D volume associated with one triangle of the source meshed point cloud. Pyramids occur for closed mesh volume calculations, since, in this case, the volume element is computed between a surface triangle and the origin of the working coordinate system. For a source depth map, the volume element is a rectangular prism. Note that instead of drawing individual volume elements, you can use M3dmetControlDraw() with M_VOLUME_ELEMENT_APPEARANCE to configure to draw only the surfaces of the source and reference objects; you can specify to draw the source object's surface, the reference object's surface, or both.

When drawing volumes, first set whether to draw all elements or a single volume element (using M3dmetControlDraw() with M_VOLUME_ELEMENT_INDEX). Then, enable the category of volume elements to draw, ether those that contributed positively to the volume calculation (M_DRAW_VOLUME_POSITIVE_ELEMENTS), those that contributed negatively (M_DRAW_VOLUME_NEGATIVE_ELEMENTS), or both (M_DRAW_VOLUME_ELEMENTS). Note that if you are drawing a single volume element and the corresponding category to which it belongs is not enabled, nothing will be drawn.

Parameters

OperationDraw3dContext3dmetId

Specifies the identifier of the draw 3D metrology context that specifies the annotations to draw and how to draw them.

> For fit results, you must set this parameter to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the draw 3D metrology context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies a predefined draw 3D metrology context. For volume results, the default context is predefined with all draw operations (M3dmetControlDraw()) set to use default settings. For fit results, this is the only possible setting. Default settings of the 3D graphics list will be used.	c		e	f	g	a		i		j	k	l	m	n	b
Draw 3D metrology context identifier	Specifies a valid draw 3D metrology context identifier, which you have allocated using M3dmetAlloc() with M_DRAW_3D_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

SrcResult3dmetId

Specifies the identifier of the fit or calculate 3D metrology result buffer, previously allocated using M3dmetAllocResult() with M_FIT_RESULT or M_CALCULATE_RESULT, respectively.

DstList3dgraId

Specifies the identifier of the 3D graphics list in which to draw. The 3D graphics list must have been previously allocated using M3dgraAlloc(), or you can specify the identifier of the 3D display's internal graphics list (inquired using M3ddispInquire() with M_3D_GRAPHIC_LIST_ID).

DstParentLabel

Specifies the label of the 3D graphic in the 3D graphics list to use as the annotation's parent.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the parent label																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ROOT_NODE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROOT_NODE	Specifies the top-most node of the 3D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the label of the 3D graphic in the 3D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Return value

Returns the label of the 3D graphic added to the 3D graphics list.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetFeature

Synopsis

Calculate scalar measurements from 3D geometries or transformation matrices.

Syntax

```
AIL_DOUBLE M3dmetFeature(
    AIL_ID Src1GeometryOrMatrix3dgeoId, //in
    AIL_ID Src2GeometryOrMatrix3dgeoId, //in
    AIL_INT64 Operation,                //in
    AIL_DOUBLE Param,                   //in
    AIL_DOUBLE *ResultPtr               //out
)
```

Description

This function calculates scalar measurements from source 3D geometry objects or transformation matrix objects.

Parameters

Src1GeometryOrMatrix3dgeoId

Specifies the first source 3D geometry object or transformation matrix object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the first source 3D geometry or transformation matrix																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of a transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

See the Parameter associations section for possible values that can be specified.

Src2GeometryOrMatrix3dgeoId

Specifies the second source 3D geometry object or transformation matrix object.

If a second source 3D geometry or transformation matrix object is not required, set this parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the second source 3D geometry																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of a transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the type of operation to perform.

See the Parameter associations section for possible values that can be specified.

Param

Specifies a scalar value for certain operations, where required. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ResultPtr

Specifies the address of the variable in which to write the results. Since the **M3dmetFeature**() function also returns the calculated value, you can set this parameter to **M_NULL**.

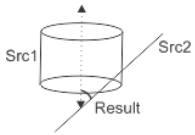
See the Parameter associations section for possible values that can be returned.

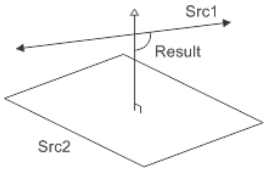
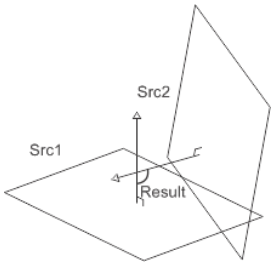
Parameter associations

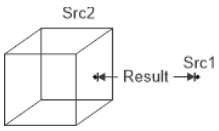
The table below lists possible values for the **Src1GeometryOrMatrix3dgeoId**, **Src2GeometryOrMatrix3dgeoId**, **Operation**, and **Param** parameters and possible values returned to the **ResultPtr** parameter.

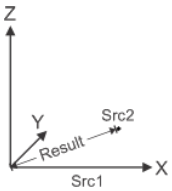
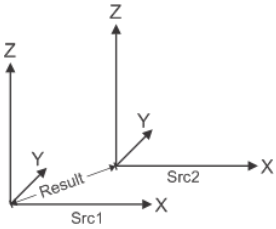
- For specifying the type of calculation to perform

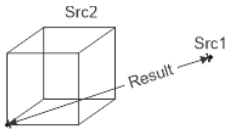
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

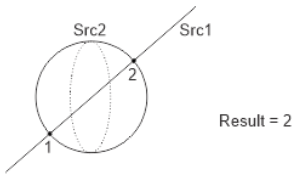
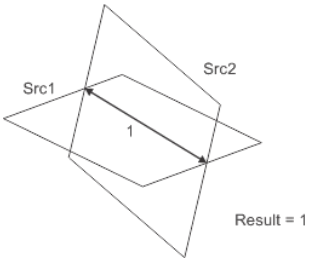
For specifying the type of calculation to perform																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Src1GeometryOrMatrix3dgeoId																
Src2GeometryOrMatrix3dgeoId																
Param																
ResultPtr - Possible values returned																
M_ANGLE	Specifies to calculate the angle between source 3D geometries.															
3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding its angle with another 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane. When the second 3D geometry is a cylinder, the angle is measured from the cylinder's central axis to the second cylinder's central axis, from 0 to 180 degrees. When the second 3D geometry is a line, the angle is measured from the cylinder's central axis to the line, from 0 to 180 degrees. When the second 3D geometry is a plane, the angle is measured from the cylinder's central axis to the plane's normal vector, from 0 to +/- 90 degrees. A negative angle is returned if the cylinder is below the plane (in the opposite direction of the plane normal vector).	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the angle between the source geometries.	c		e	f	g	a		i		j	k	l	m	n	b

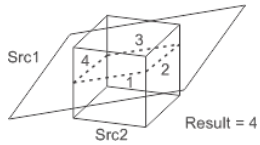
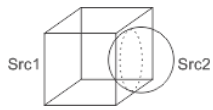
3D line geometry object identifier	Specifies the identifier of the 3D line geometry object, for finding its angle with another 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane. When the second 3D geometry is a cylinder, the angle is measured from the line to the cylinder's central axis, from 0 to 180 degrees. When the second 3D geometry is a line, the angle is measured from the line to the second line, from 0 to 180 degrees. When the second 3D geometry is a plane, the angle is measured from the line to the plane's normal vector, from 0 to +/- 90 degrees. A negative angle is returned if the cylinder is below the plane (in the opposite direction of the plane normal vector).	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the angle between the source geometries.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier	Specifies the identifier of the 3D plane geometry object, for finding its angle with another 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane. When the second 3D geometry is a cylinder, the angle is measured from the plane's normal vector to the cylinders central axis, from 0 to +/- 90 degrees. A negative angle is returned if the cylinder is below the plane (in the opposite direction of the plane's normal vector). When the second 3D geometry is a line, the angle is measured from the plane's normal vector to the line, from 0 to +/- 90 degrees. A negative angle is returned if the cylinder is below the plane (in the opposite direction of the plane's normal vector). When the second 3D geometry is a plane, the angle is measured from the plane's normal vector to the second plane's normal vector, from 0 to 180 degrees.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the angle between the source geometries.	c		e	f	g	a		i		j	k	l	m	n	b

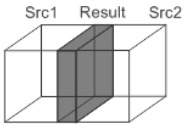
Transformation matrix object identifier	Specifies the identifier of the transformation matrix, for finding its angle with another transformation matrix. The transformation matrix must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. You will obtain the angle between the two rotation matrices. Note that the two matrices must be similarity transformations, and must have the same handedness. That is, if you call M3dgeoInquire() with M_SIMILARITY, it must return M_TRUE, and if you call M3dgeoInquire() with M_MIRROR, it must return the same value for both matrices.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second transformation matrix, previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the angle between the source matrices.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLIP	Specifies to calculate if a 3D line geometry object is inside, outside, or partially inside a second 3D geometry. Note that, if the second geometry is a plane, inside is the same side as the plane's normal vector. To inquire this vector, use M3dgeoInquire() with M_NORMAL_....	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D line geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Sets the length of the 3D line geometry, when clipping an infinite line with a plane. If Src2GeometryOrMatrix3dgeoId is not a plane, you must set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the length of the 3D line.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns whether the line is inside of the second source 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
M_INSIDE	Specifies that the line is completely contained within the second source geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTSIDE	Specifies that the line is not contained within the second source geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARTIALLY_INSIDE	Specifies that the line is partially contained within the second source geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_POINT	Specifies to calculate the distance between the first specified 3D geometry (which must be a point) and the closest point on the second specified geometry. 	c		e	f	g	a		i		j	k	l	m	n	b
3D point geometry object identifier	Specifies the identifier of a 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b

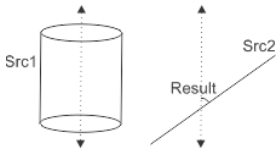
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr	Returns the distance between the two points.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE		Specifies to calculate the distance between two 3D points, the origins of two transformation matrices, or the origin of a transformation matrix and a 3D point.	c		e	f	g	a		i		j	k	l	m	n	b
	3D point geometry object identifier - distance between points	Specifies the identifier of a 3D point geometry object, for finding the distance between it and another point. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point. 	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr	Returns the distance between the points.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier - distance between coordinate system origin and point	Specifies the identifier of a transformation matrix object that defines a coordinate system, for finding the distance between its origin and a point. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.  	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr	Returns the distance between the origin and the point.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier - distance between coordinate system origins	Specifies the identifier of a transformation matrix object that defines a coordinate system, for finding the distance between its origin and the origin of another coordinate system. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.  	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D transformation matrix, previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr	Returns the distance between the origins.	c		e	f	g	a		i		j	k	l	m	n	b

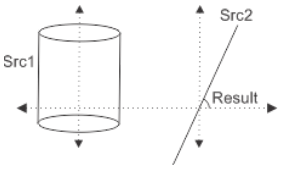
M_EXTRUSION_BORDER	Specifies to calculate whether extruding a box face to a plane would succeed, if extruding until a corner of the closest face meets the plane. This operation evaluates the specified 3D box and plane geometries to determine if the box face that is most parallel and closest to the plane can be extruded until the face's closest corner meets the plane. If the most parallel face and its opposite both intersect the plane, there is no meaningful way to extrude the box. In this case, the operation returns M_FAIL.	c		e	f	g	a		i		j	k	l	m	n	b
3D box geometry object identifier	Specifies the identifier of a 3D box geometry object, for determining whether it can be extruded to a plane. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns whether the extrusion operation would succeed.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies that the extrusion operation would not complete successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the extrusion operation would complete successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRUSION_CENTER	Specifies to calculate whether extruding a box face to a plane would succeed, if extruding until the center of the closest face meets the plane. This operation evaluates the specified 3D box and plane geometries to determine if the box face that is most parallel and closest to the plane can be extruded until the center of the face touches the plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D box geometry object identifier	Specifies the identifier of a 3D box geometry object, for determining whether it can be extruded to a plane. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns whether the extrusion operation would succeed.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies that the extrusion operation would not complete successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the extrusion operation would complete successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_FARTHEST_POINT	Specifies to calculate the distance between the first specified 3D geometry (which must be a point) and the farthest point on the second specified geometry.  	c		e	f	g	a		i		j	k	l	m	n	b

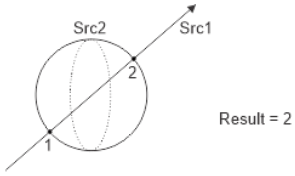
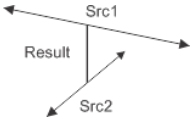
3D point geometry object identifier	Specifies the identifier of a 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined. Note that infinite cylinders and lines are not supported.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the distance between the two points.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERSECTION	Specifies to calculate the number of intersections between source 3D geometries; the value returned is the number of points or lines that constitute the intersection(s).	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier - intersection points	Specifies the identifier of the 3D line geometry object, for finding its intersection with a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. You will obtain the number of intersection points, either 0, 1, or 2.  For an intersection operation that treats the source 3D line as a ray (extending infinitely in one direction), use M_RAY_CAST.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the number of intersection points.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier - intersection line	Specifies the identifier of a 3D plane geometry object, for finding its intersection with a second plane. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. You will obtain the number of intersection lines, either 0 or 1.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the number of intersection lines.	c		e	f	g	a		i		j	k	l	m	n	b

3D plane geometry object identifier - intersection polygon	Specifies the identifier of the 3D plane geometry object, for finding its intersection with a box. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. You will obtain the number of intersection lines, either 0, 3, 4, 5, or 6.  	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c	e	f	g	a	i	j	k	l	m	n	b
ResultPtr	Returns the number of intersection lines.	c	e	f	g	a	i	j	k	l	m	n	b
M_IS_INSIDE	Specifies to calculate if a first source 3D geometry is inside, outside, or partially inside a second 3D geometry. For the two source 3D geometries in the following image, the function returns M_PARTIALLY_INSIDE.  	c	e	f	g	a	i	j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of the first 3D geometry object, for calculating if it is inside, outside, or partially inside the second 3D geometry object. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, line, point, or sphere.	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, sphere.	c	e	f	g	a	i	j	k	l	m	n	b
ResultPtr	Returns whether the first source 3D geometry is inside of the second source 3D geometry.	c	e	f	g	a	i	j	k	l	m	n	b
M_INSIDE	Specifies that the first source geometry object is completely contained within the second source geometry object.	c	e	f	g	a	i	j	k	l	m	n	b
M_OUTSIDE	Specifies that the first source geometry object is not contained within the second source geometry object.	c	e	f	g	a	i	j	k	l	m	n	b
M_PARTIALLY_INSIDE	Specifies that the first source geometry object is partially contained within the second source geometry object.	c	e	f	g	a	i	j	k	l	m	n	b
M_OVERLAP	Specifies to calculate the percentage of volume overlap between source 3D geometries. The percentage is calculated according to the following formula, where $V_{intersection}$ is the overlapping volume and V_{union} is the combined 3D geometry volume: $(V_{intersection} / V_{union}) * 100$ Set the resolution of the calculation with Param, which sets the smallest cubic unit with which to calculate the volume. Note that infinite cylinders are not supported.	c	e	f	g	a	i	j	k	l	m	n	b

3D box geometry object identifier	Specifies the identifier of a 3D box geometry object, for finding the percentage of volume overlap between the box and another 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box. 	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, finite cylinder, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the size of the smallest measurable detail. The specified value represents the length of a cube, for which the operation can resolve when calculating the geometries' volumes. Smaller values relative to the dimensions of the source 3D geometries will take longer to compute, but produce more accurate results. Note that the accuracy of M_OVERLAP decreases linearly with this parameter. The time required to perform this operation increases quadratically with a high overlap percentage, and cubically with a low overlap percentage.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the resolution, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the percentage of volume overlap.	c		e	f	g	a		i		j	k	l	m	n	b
3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding the percentage of volume overlap between the cylinder and another 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder. Note that infinite cylinders are not supported for this operation.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, finite cylinder, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the size of the smallest measurable detail. The specified value represents the length of a cube, for which the operation can resolve when calculating the geometries' volumes. Smaller values relative to the dimensions of the source 3D geometries will take longer to compute, but produce more accurate results. Note that the accuracy of M_OVERLAP decreases linearly with this parameter. The time required to perform this operation increases quadratically with a high overlap percentage, and cubically with a low overlap percentage.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the resolution, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the percentage of volume overlap.	c		e	f	g	a		i		j	k	l	m	n	b
3D sphere geometry object identifier	Specifies the identifier of a 3D sphere geometry object, for finding the percentage of volume overlap between the sphere and another 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, finite cylinder, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b

Param	Specifies the size of the smallest measurable detail. The specified value represents the length of a cube, for which the operation can resolve when calculating the geometries' volumes. Smaller values relative to the dimensions of the source 3D geometries will take longer to compute, but produce more accurate results. Note that the accuracy of M_OVERLAP decreases linearly with this parameter. The time required to perform this operation increases quadratically with a high overlap percentage, and cubically with a low overlap percentage.	c	e	f	g	a	i	j	k	l	m	n	b
Value > 0.0	Specifies the resolution, in world units.	c	e	f	g	a	i	j	k	l	m	n	b
ResultPtr	Returns the percentage of volume overlap.	c	e	f	g	a	i	j	k	l	m	n	b
M_PARALLELISM	Specifies to calculate the angular deviation from a parallel orientation for source 3D geometries. For a 3D cylinder, the angle is measured from the cylinder's central axis. For a 3D plane, the angle is measured from the plane's normal vector.	c	e	f	g	a	i	j	k	l	m	n	b
3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding the degree to which it is parallel to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the cylinder is parallel to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).  	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c	e	f	g	a	i	j	k	l	m	n	b
ResultPtr	Returns the deviation from a parallel orientation, in degrees.	c	e	f	g	a	i	j	k	l	m	n	b
3D line geometry object identifier	Specifies the identifier of a 3D line geometry object, for finding the degree to which it is parallel to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the line is parallel to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c	e	f	g	a	i	j	k	l	m	n	b
ResultPtr	Returns the deviation from a parallel orientation, in degrees.	c	e	f	g	a	i	j	k	l	m	n	b
3D plane geometry object identifier	Specifies the identifier of a 3D plane geometry object, for finding the degree to which it is parallel to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the plane is parallel to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c	e	f	g	a	i	j	k	l	m	n	b

	ResultPtr	Returns the deviation from a parallel orientation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_PERPENDICULARITY		Specifies to calculate the angular deviation from a perpendicular orientation for source 3D geometries. For a 3D cylinder, the angle is measured from the cylinder's central axis. For a 3D plane, the angle is measured from the plane's normal vector.	c		e	f	g	a		i		j	k	l	m	n	b
	3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding the degree to which it is perpendicular to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the cylinder is perpendicular to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).  	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr	Returns the deviation from a perpendicular orientation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	3D line geometry object identifier	Specifies the identifier of a 3D line geometry object, for finding the degree to which it is perpendicular to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the line is perpendicular to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr	Returns the deviation from a perpendicular orientation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	3D plane geometry object identifier	Specifies the identifier of a 3D plane geometry object, for finding the degree to which it is perpendicular to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the plane is perpendicular to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultPtr	Returns the deviation from a perpendicular orientation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_RAY_CAST	Specifies to calculate the number of intersection points between a ray and a second source 3D geometry, where a ray is a line that extends infinitely in one direction from the line's start point. 	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies the identifier of the 3D line geometry object that defines the ray, for finding its intersection with a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. You will obtain the number of intersection points, either 0, 1, or 2.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the number of intersection points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SHORTEST_LINE	Specifies to calculate the length of the shortest line between two source 3D line geometries. 	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies the identifier of the first 3D line geometry object. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
ResultPtr	Returns the shortest distance between the two lines.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_DOUBLE*. If the requested information does not fit into an *AIL_DOUBLE*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetFeatureEx

Synopsis

Calculates geometric and scalar measurements from 3D geometries or transformation matrices.

Syntax

```
void M3dmetFeatureEx(
    AIL_ID Feature3dmetContextId,       //in
    AIL_ID Src1GeometryOrMatrix3dgeoId, //in
    AIL_ID Src2GeometryOrMatrix3dgeoId, //in
    AIL_ID Src3Geometry3dgeoId,         //in
    AIL_ID Result3dmetOr3dgeoId,        //out
    AIL_INT64 Operation,                //in
    AIL_DOUBLE Param,                   //in
    AIL_INT64 ControlFlag               //in
)
```

Description

This function calculates geometric and scalar measurements from source 3D geometry or transformation matrix objects. To more easily compute certain scalar measurements, use M3dmetFeature().

Parameters

Feature3dmetContextId

Reserved for future expansion and must be set to **M_DEFAULT**.

Src1GeometryOrMatrix3dgeoId

Specifies the first source 3D geometry or transformation matrix object.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the first source 3D geometry or transformation matrix object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b
Transformation matrix object identifier	Specifies the identifier of a transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

See the Parameter associations section for possible values that can be specified.

Src2GeometryOrMatrix3dgeoId

Specifies the second source 3D geometry or transformation matrix object.

If a second source 3D geometry or transformation matrix object is not required, set this parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the second source 3D geometry or transformation matrix object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b

Transformation matrix object identifier	Specifies the identifier of a transformation matrix object. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b

See the Parameter associations section for possible values that can be specified.

Src3Geometry3dgeoId

Specifies the third source 3D geometry object.

If a third source 3D geometry object is not required, set this parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the third source 3D geometry																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and must have been successfully defined.	c		e	f	g	a		i		j	k	l	m	n	b

See the Parameter associations section for possible values that can be specified.

Result3dmetOr3dgeoId

Specifies the identifier of the calculate 3D metrology result buffer in which to store the results of the calculate operation, or specifies the 3D geometry object in which to define a resulting geometry.

The result buffer must have been previously allocated using M3dmetAllocResult() with M_CALCULATE_RESULT. To retrieve results for the calculate operation, use M3dmetGetResult().

If you specify a 3D geometry object, a successful calculate operation results in a defined 3D geometry. If the specified operation can never produce a geometry, you will receive an error. If you pass a 3D geometry object and the operation can produce geometries but did not with the given source 3D geometry objects, the resulting 3D geometry object will be of type M_NOT_INITIALIZED.

Operation

Specifies the type of operation to perform.

See the Parameter associations section for possible values that can be specified.

Param

Specifies a scalar value for certain operations, where required. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

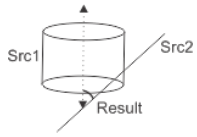
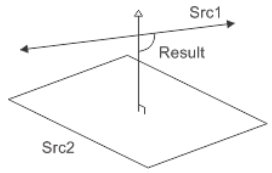
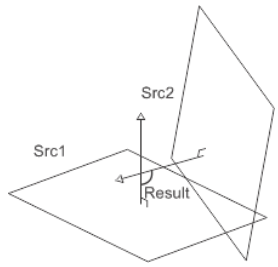
Parameter associations

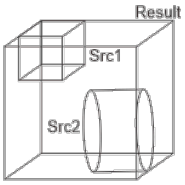
The table below lists possible values for the **Src1GeometryOrMatrix3dgeoId**, **Src2GeometryOrMatrix3dgeoId**, **Src3Geometry3dgeoId**, **Operation**, and **Param** parameters.

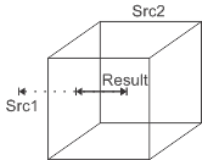
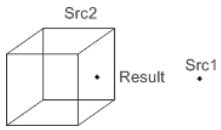
- For specifying the type of calculation to perform

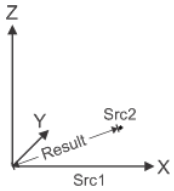
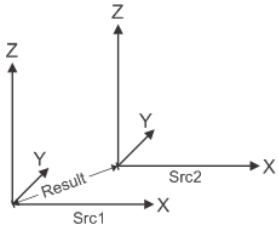
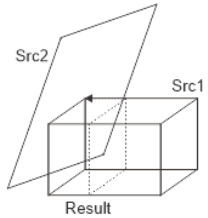
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

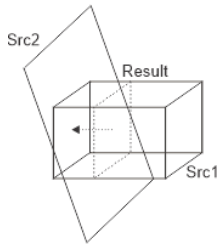
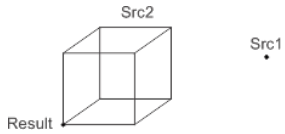
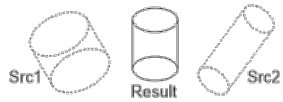
For specifying the type of calculation to perform																
Operation			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Src1GeometryOrMatrix3dgeoId																	
		Src2GeometryOrMatrix3dgeoId																
		Src3Geometry3dgeoId																
		Param																

M_ANGLE	Specifies to calculate the angle between source 3D geometries. You can retrieve results for this operation using M3dmetGetResult() with M_ANGLE.														
3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding its angle with another 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane. When the second 3D geometry is a cylinder, the angle is measured from the cylinder's central axis to the second cylinder's central axis, from 0 to 180 degrees. When the second 3D geometry is a line, the angle is measured from the cylinder's central axis to the line, from 0 to 180 degrees. When the second 3D geometry is a plane, the angle is measured from the cylinder's central axis to the plane's normal vector, from 0 to +/- 90 degrees. A negative angle is returned if the cylinder is below the plane (in the opposite direction of the plane normal vector).	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies the identifier of the 3D line geometry object, for finding its angle with another 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane. When the second 3D geometry is a cylinder, the angle is measured from the line to the cylinder's central axis, from 0 to 180 degrees. When the second 3D geometry is a line, the angle is measured from the line to the second line, from 0 to 180 degrees. When the second 3D geometry is a plane, the angle is measured from the line to the plane's normal vector, from 0 to +/- 90 degrees. A negative angle is returned if the cylinder is below the plane (in the opposite direction of the plane normal vector).	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier	Specifies the identifier of the 3D plane geometry object, for finding its angle with another 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.  	c		e	f	g	a		i		j	k	l	m	n	b

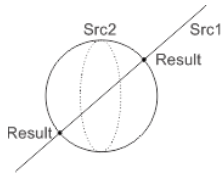
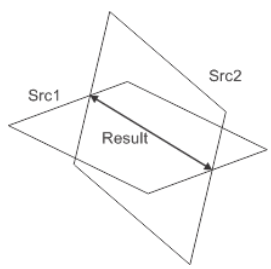
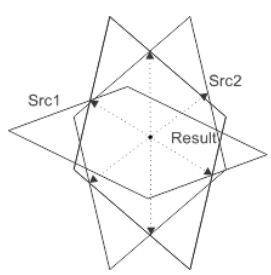
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane. When the second 3D geometry is a cylinder, the angle is measured from the plane's normal vector to the cylinders central axis, from 0 to +/- 90 degrees. A negative angle is returned if the line is below the plane (in the opposite direction of the plane's normal vector). When the second 3D geometry is a line, the angle is measured from the plane's normal vector to the line, from 0 to +/- 90 degrees. A negative angle is returned if the cylinder is below the plane (in the opposite direction of the plane's normal vector). When the second 3D geometry is a plane, the angle is measured from the plane's normal vector to the second plane's normal vector, from 0 to 180 degrees.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier	Specifies the identifier of the transformation matrix, for finding its angle with another transformation matrix. The transformation matrix must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX. You will obtain the angle of the axis-angle rotation between the matrices. This is the angle through which an object with the Src1 rotation must be rotated to align with an object with the Src2 rotation. Note that the two matrices must be similarity transformations, and must have the same handedness. That is, if you call M3dgeoInquire() with M_SIMILARITY, it must return M_TRUE, and if you call M3dgeoInquire() with M_MIRROR, it must return the same value for both matrices.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second transformation matrix, previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX		Specifies to calculate the axis-aligned bounding box of source 3D geometries.  You can specify geometries of type M_NOT_INITIALIZED for Src2GeometryOrMatrix3dgeoId. If not initialized, this geometry has no effect on the result. This allows you to specify the same geometry for Src2GeometryOrMatrix3dgeoId and Result3dmetOr3dgeoId without previously initializing the result 3D geometry object, which is useful when using a loop to compute the bounding box of many geometries. Note that infinite geometries are not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	3D geometry object identifier	Specifies the identifier of a 3D geometry object, for finding its axis-aligned bounding box, or the combined axis-aligned bounding box with a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, finite cylinder, finite line, point, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of a second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY. The 3D geometry can be either defined as a box, finite cylinder, finite line, point, or sphere, or of type M_NOT_INITIALIZED. If not required, set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

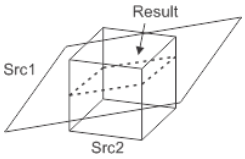
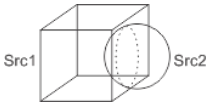
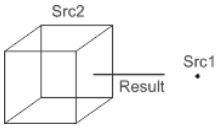
M_CLIP	Specifies to truncate a 3D line geometry object such that it lies inside a second 3D geometry. Note that, if the second geometry is a plane, inside is the same side as the plane's normal vector. To inquire this vector, use M3dgeoInquire() with M_NORMAL_....  You can retrieve results for this operation using M3dmetGetResult() with M_IS_INSIDE.	c	e	f	g	a	i	j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D line geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. If the line is inside the second specified 3D geometry object (Src2GeometryOrMatrix3dgeoId), you will obtain a clipped line; you will obtain a geometry of type M_NOT_INITIALIZED if the line does not fall inside the second 3D geometry.	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c	e	f	g	a	i	j	k	l	m	n	b
Param	Sets the length of the clipped line, when clipping an infinite line with a plane. If Src2GeometryOrMatrix3dgeoId is not a plane, you must set this parameter to **M_DEFAULT**.	c	e	f	g	a	i	j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c	e	f	g	a	i	j	k	l	m	n	b
Value >= 0.0	Specifies the length of the clipped line.	c	e	f	g	a	i	j	k	l	m	n	b
M_CLOSEST_POINT	Specifies to calculate the point on the second specified 3D geometry that is closest to the first specified 3D geometry, which must be a point. This operation also calculates the distance between the points.  To retrieve the calculated distance, use M3dmetGetResult() with M_DISTANCE.	c	e	f	g	a	i	j	k	l	m	n	b
3D point geometry object identifier	Specifies the identifier of a 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point. You will obtain the point on the second 3D geometry that is closest to this point.	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined.	c	e	f	g	a	i	j	k	l	m	n	b
M_DISTANCE	Specifies to calculate the distance between source 3D geometries. You can retrieve results for this operation using M3dmetGetResult() with M_DISTANCE.	c	e	f	g	a	i	j	k	l	m	n	b
3D point geometry object identifier - distance between points	Specifies the identifier of the 3D point geometry object, for finding the distance between it and another point. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point. 	c	e	f	g	a	i	j	k	l	m	n	b

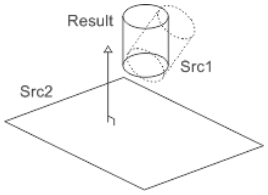
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier - distance between coordinate system origin and point	Specifies the identifier of a transformation matrix object that defines a coordinate system, for finding the distance between its origin and a point. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.  	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation matrix object identifier - distance between coordinate system origins	Specifies the identifier of a transformation matrix object that defines a coordinate system, for finding the distance between its origin and the origin of another coordinate system. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.  	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D transformation matrix, previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRUSION_BORDER		Specifies to lengthen or shorten a 3D box geometry relative to a specified plane; Aurora Imaging Library extrudes the box face that is most parallel and closest to the plane until the face's closest corner meets the plane.  You can retrieve whether this operation was successful using M3dmetGetResult() with M_STATUS_EXTRUSION. If the most parallel face and its opposite both intersect the plane, there is no meaningful way to extrude the box. In this case, the specified 3D box geometry is copied directly into the result object, and M3dmetGetResult() with M_STATUS_EXTRUSION returns M_FAIL.	c		e	f	g	a		i		j	k	l	m	n	b
	3D box geometry object identifier	Specifies the identifier of a 3D box geometry object, for extruding it to a plane 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b

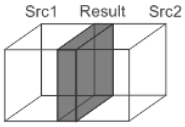
M_EXTRUSION_CENTER	Specifies to lengthen or shorten a 3D box geometry relative to a specified plane; Aurora Imaging Library extrudes the box face that is most parallel and closest to the plane until the center of the face touches the plane.  You can retrieve whether this operation was successful using M3dmetGetResult() with M_STATUS_EXTRUSION.	c	e	f	g	a	i	j	k	l	m	n	b
3D box geometry object identifier	Specifies the identifier of a 3D box geometry object, for extruding it to a plane 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c	e	f	g	a	i	j	k	l	m	n	b
M_FARTHEST_POINT	Specifies to calculate the point on the second specified 3D geometry that is farthest from the first specified 3D geometry, which must be a point. This operation also calculates the distance between the points.  To retrieve the calculated distance, use M3dmetGetResult() with M_DISTANCE.	c	e	f	g	a	i	j	k	l	m	n	b
3D point geometry object identifier	Specifies the identifier of a 3D point geometry object, for finding the point on the second specified 3D geometry that is farthest from the first source point. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point. You will obtain the point on the second source geometry object that is the farthest distance from this point.	c	e	f	g	a	i	j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined. > Note that infinite cylinders and lines are not supported.	c	e	f	g	a	i	j	k	l	m	n	b
M_INTERPOLATION	Specifies to calculate the interpolation between source 3D geometries or matrices. Both sources must be the same type of 3D geometry, or they must both be transformation matrix objects.  You must specify an interpolation factor (Param), where 0.0 maps to the first source 3D geometry, and 1.0 maps to the second source 3D geometry. Values between 0.0 and 1.0 are interpolated, and values outside of this range are extrapolated. If extrapolation would result in a negative scale, the scale is set to 0. This calculation is a linear interpolation between source objects. In general, given Src1, Src2, and an interpolation factor, the resulting object's size will be: Factor*Src1 + (1-Factor)*Src2.	c	e	f	g	a	i	j	k	l	m	n	b

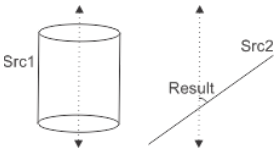
	3D geometry object identifier - interpolate between geometries	Specifies the identifier of the first of two 3D geometry objects of the same type, for finding the interpolation. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined successfully. Note that, for 3D lines and cylinders, both source geometries must be finite or infinite.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined. The second source 3D geometry must be the same type as the first source 3D geometry. > Note that, for 3D lines and cylinders, both source geometries must be finite or infinite.	c		e	f	g	a		i		j	k	l	m	n	b
	Param	Specifies the interpolation factor.	c		e	f	g	a		i		j	k	l	m	n	b
	Transformation object identifier - interpolate between transformation matrices	Specifies the identifier of the first of two transformation matrix objects, for finding the interpolation. The transformation matrix object must have been previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX, and defined as a similarity transformation.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source transformation matrix object, previously allocated using M3dgeoAlloc() with M_TRANSFORMATION_MATRIX, and defined as a similarity transformation. > Note that the two matrices must be similarity transformations, and must have the same handedness. That is, if you call M3dgeoInquire() with M_SIMILARITY, the function returns M_TRUE, and if you call M3dgeoInquire() with M_MIRROR, the function returns the same value, for both matrices.	c		e	f	g	a		i		j	k	l	m	n	b
	Param	Specifies the interpolation factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERSECTION		Specifies to calculate the intersection line, points, or polygon between source 3D geometries. You can retrieve the number of resulting 3D geometries for this operation using M3dmetGetResult() with M_NUMBER. If all source 3D geometries are planes, you can retrieve the parallelism between the planes, using M3dmetGetResult() with M_ANGLE_TO_EDGE_CASE; you can also retrieve the parallelism between a source line and plane. Note that, for three source planes, M_ANGLE_TO_EDGE_CASE returns the parallelism of the two planes that are most parallel, where 0 means completely parallel.	c		e	f	g	a		i		j	k	l	m	n	b

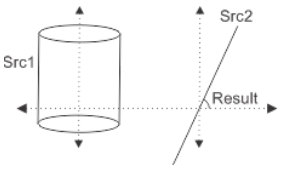
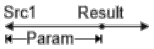
3D line geometry object identifier - intersection points	Specifies the identifier of the 3D line geometry object, for finding its intersection with a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. If the line intersects the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId), you can obtain 1 or 2 intersection points; you will obtain only 1 point if the second geometry is a plane or if the line is finite and does not cross the second geometry entirely. If the line does not intersect the second specified 3D geometry, you will obtain a geometry of type M_NOT_INITIALIZED.  For an intersection operation that treats the source 3D line as a ray (extending infinitely in one direction), use M_RAY_CAST.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier - intersection line	Specifies the identifier of a 3D plane geometry object, for finding its intersection with a second plane. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. If the plane intersects the second specified plane (Src2GeometryOrMatrix3dgeoId), you will obtain 1 infinite intersection line.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier - intersection point	Specifies the identifier of a 3D plane geometry object, for finding its intersection with a second and third plane. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. If the plane intersects the second and third specified planes (Src2GeometryOrMatrix3dgeoId and Src3Geometry3dgeoId, respectively), you will obtain 1 intersection point. If the plane does not intersect the second and third specified planes, you will obtain a geometry of type M_NOT_INITIALIZED.  	c		e	f	g	a		i		j	k	l	m	n	b

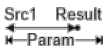
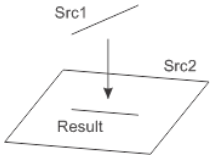
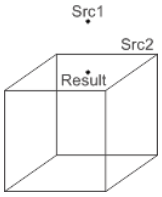
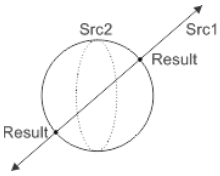
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
	Src3Geometry3dgeoId	Specifies the identifier of the third source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
	3D plane geometry object identifier - intersection polygon	Specifies the identifier of the 3D plane geometry object, for finding its intersection with a 3D box geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. If the plane intersects the specified box (Src2GeometryOrMatrix3dgeoId), you will obtain a polygon consisting of 3 to 6 finite intersection lines. 	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_INSIDE		Specifies to calculate if a first source 3D geometry is inside, outside, or partially inside a second 3D geometry. For the two source 3D geometries in the following image, the calculated result is M_PARTIALLY_INSIDE.  You can retrieve results for this operation using M3dmetGetResult() with M_IS_INSIDE.	c		e	f	g	a		i		j	k	l	m	n	b
	3D geometry object identifier	Specifies the identifier of the first 3D geometry object, for calculating if it is inside, outside, or partially inside the second 3D geometry object. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, line, point, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL_AT_POSITION		Specifies to calculate a unit normal line (one unit in length) on a source 3D geometry closest to a given point. 	c		e	f	g	a		i		j	k	l	m	n	b
	3D point geometry object identifier	Specifies the identifier of a 3D point geometry object, for calculating a unit line along an outward facing normal direction that is in line with the point, with a start point at the surface of a second specified 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point. Note that the point can be inside or outside the second 3D geometry; the calculated unit line is always directed outwards from the second geometry's surface.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b

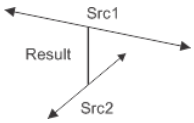
M_ORTHOGONALIZE	Specifies to rotate the specified source 3D geometry such that it is arranged orthogonally with another 3D geometry. You can specify a point (Src3Geometry3dgeoId) about which to perform the rotation. If you do not specify a point, the rotation is performed around the first specified 3D geometry's start point, or, if the first 3D geometry is a plane, around the origin.	c		e	f	g	a		i		j	k	l	m	n	b
3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for rotation to an orthogonal position with respect to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
Src3Geometry3dgeoId	Specifies the identifier of an optional 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined. If specified, the M_ORTHOGONALIZE operation is performed around the point. Set this parameter to **M_NULL** if not used.	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies the identifier of a 3D line geometry object, for rotation to an orthogonal position with respect to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
Src3Geometry3dgeoId	Specifies the identifier of an optional 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined. If specified, the M_ORTHOGONALIZE operation is performed around the point. Set this parameter to **M_NULL** if not used.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier	Specifies the identifier of a 3D plane geometry object, for rotation to an orthogonal position with respect to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
Src3Geometry3dgeoId	Specifies the identifier of an optional 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and successfully defined. If specified, the M_ORTHOGONALIZE operation is performed around the point. Set this parameter to **M_NULL** if not used.	c		e	f	g	a		i		j	k	l	m	n	b

M_OVERLAP	Specifies to calculate the percentage of volume overlap between source 3D geometries. The percentage is calculated according to the following formula, where $V_{intersection}$ is the overlapping volume and V_{union} is the combined 3D geometry volume: $(V_{intersection} / V_{union}) * 100$ Set the resolution of the calculation with Param, which sets the smallest cubic unit with which to calculate the volume. Note that infinite cylinders are not supported. You can retrieve results for this operation using M3dmetGetResult() with M_OVERLAP. You can also use M3dmetGetResult() to retrieve the volume of the union of the two source 3D geometries (M_UNION), the volume of their intersection (M_INTERSECTION), and the difference between the union and intersection (M_UNION - M_INTERSECTION) of the two source 3D geometries (M_DIFFERENCE).	c		e	f	g	a		i		j	k	l	m	n	b
3D box geometry object identifier	Specifies the identifier of a 3D box geometry object, for finding the percentage of volume overlap between the box and another 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, finite cylinder, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the size of the smallest measurable detail. The specified value represents the length of a cube, for which the operation can resolve when calculating the geometries' volumes. Smaller values relative to the dimensions of the source 3D geometries will take longer to compute, but produce more accurate results. Note that the accuracy of M_OVERLAP decreases linearly with this parameter. The time required to perform this operation increases quadratically with a high overlap percentage, and cubically with a low overlap percentage.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the resolution, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding the percentage of volume overlap between the cylinder and another 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder. Note that infinite cylinders are not supported for this operation.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, finite cylinder, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the size of the smallest measurable detail. The specified value represents the length of a cube, for which the operation can resolve when calculating the geometries' volumes. Smaller values relative to the dimensions of the source 3D geometries will take longer to compute, but produce more accurate results. Note that the accuracy of M_OVERLAP decreases linearly with this parameter. The time required to perform this operation increases quadratically with a high overlap percentage, and cubically with a low overlap percentage.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the resolution, in world units.	c		e	f	g	a		i		j	k	l	m	n	b

3D sphere geometry object identifier	Specifies the identifier of a 3D sphere geometry object, for finding the percentage of volume overlap between the sphere and another 3D geometry. The 3D geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, finite cylinder, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the size of the smallest measurable detail. The specified value represents the length of a cube, for which the operation can resolve when calculating the geometries' volumes. Smaller values relative to the dimensions of the source 3D geometries will take longer to compute, but produce more accurate results. Note that the accuracy of M_OVERLAP decreases linearly with this parameter. The time required to perform this operation increases quadratically with a high overlap percentage, and cubically with a low overlap percentage.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the resolution, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARALLELISM	Specifies to calculate the angular deviation from a parallel orientation for source 3D geometries. For a 3D cylinder, the angle is measured from the cylinder's central axis. For a 3D plane, the angle is measured from the plane's normal vector. You can retrieve results for this operation using M3dmetGetResult() with M_PARALLELISM.	c		e	f	g	a		i		j	k	l	m	n	b
3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding the degree to which it is parallel to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the cylinder is parallel to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).  	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies the identifier of a 3D line geometry object, for finding the degree to which it is parallel to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the line is parallel to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier	Specifies the identifier of a 3D plane geometry object, for finding the degree to which it is parallel to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the plane is parallel to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c		e	f	g	a		i		j	k	l	m	n	b

	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_PERPENDICULARITY		Specifies to calculate the angular deviation from a perpendicular orientation for source 3D geometries. For a 3D cylinder, the angle is measured from the cylinder's central axis. For a 3D plane, the angle is measured from the plane's normal vector. You can retrieve results for this operation using M3dmetGetResult() with M_PERPENDICULARITY.	c		e	f	g	a		i		j	k	l	m	n	b
	3D cylinder geometry object identifier	Specifies the identifier of a 3D cylinder geometry object, for finding the degree to which it is perpendicular to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the cylinder is perpendicular to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).  	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
	3D line geometry object identifier	Specifies the identifier of a 3D line geometry object, for finding the degree to which it is perpendicular to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the line is perpendicular to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
	3D plane geometry object identifier	Specifies the identifier of a 3D plane geometry object, for finding the degree to which it is perpendicular to a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. You will obtain an angle from 0 to 90 degrees, where 0 indicates that the plane is perpendicular to the second specified 3D geometry (Src2GeometryOrMatrix3dgeoId).	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder, line, or plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT_ON_LINE		Specifies to calculate a 3D point geometry on the source 3D line geometry at a specified distance from the line's start point.  	c		e	f	g	a		i		j	k	l	m	n	b
	3D line geometry object identifier	Specifies the identifier of a 3D line geometry object, on which to place the calculated 3D point. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
	Param	Specifies the distance from the start point of the 3D line at which to create the 3D point.	c		e	f	g	a		i		j	k	l	m	n	b

M_POINT_ON_LINE_CLIPPED	Specifies to calculate a 3D point geometry on the source 3D line geometry at a specified distance from the line's start point. If the specified distance would place the point outside the limits of a finite line, the point is clipped onto the line's end point or start point, whichever is closer. 	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies the identifier of a 3D line geometry object, on which to calculate the 3D point. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the distance from the start point of the line, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJECTION	Specifies to calculate a point or line 3D geometry from a projection of a source point or line onto the surface of a second source 3D geometry. You can project a line onto a 3D plane only.	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier - project line	Specifies the identifier of a 3D line geometry object, for projecting onto a 3D plane geometry. The 3D line geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line. 	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D point geometry object identifier - project point	Specifies the identifier of a 3D point geometry object, for projecting onto a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point. 	c		e	f	g	a		i		j	k	l	m	n	b
Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, line, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
M_RAY_CAST	Specifies to calculate the intersection point(s) between a ray and a second source 3D geometry, where a ray is a line that extends infinitely in one direction from the line's start point.  You can retrieve results for this operation using M3dmetGetResult() with M_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b

	3D line geometry object identifier	Specifies the identifier of the 3D line geometry object that defines the ray, for finding its intersection point(s) with a second 3D geometry. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box, cylinder, plane, or sphere.	c		e	f	g	a		i		j	k	l	m	n	b
M_SHORTEST_LINE		Specifies to calculate the shortest line between two source 3D line geometries. This operation also calculates the length of the resulting line.  To retrieve the resulting line's length, use M3dmetGetResult() with M_DISTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
	3D line geometry object identifier	Specifies the identifier of the first 3D line geometry object. The 3D geometry must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2GeometryOrMatrix3dgeoId	Specifies the identifier of the second source 3D geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetFit

Synopsis

Fits a 3D geometry to a point cloud or depth map.

Syntax

```
void M3dmetFit(
    AIL_ID Context3dmetId,           //in
    AIL_ID SrcContainerOrImageBufId, //in
    AIL_INT GeometryType,            //in
    AIL_ID Result3dmetOr3dgeoId,     //out
    AIL_DOUBLE OutlierDistance,      //in
    AIL_INT64 ControlFlag            //in
)
```

Description

This function fits a 3D geometry to a point cloud or depth map using an iterative fit operation. The function starts from an initial fit estimate of the size and position of the 3D geometry. The function then internally iterates through better estimates of the correct size and position of the 3D geometry until a specified stop condition is reached. Upon each iteration, the function calculates the average root-mean-square (RMS) error per inlier, until a stop condition is met.

The initial fit estimate is calculated according to the mode specified using M3dmetControl() with M_ESTIMATION_MODE. A set of inlier points is then defined according to OutlierDistance, and a new size and placement for the 3D geometry, which reduces the average RMS error per inlier, is calculated. This process then repeats until a stop condition is met.

If the initial fit estimate mode is set to M_FROM_GEOMETRY, the fit operation will use the specified geometry in the fit 3D metrology context. By default, the context contains an undefined geometry. If you try to initiate the fit operation and use the undefined geometry to determine an initial fit estimate, an error will occur. You can use M3dmetCopy() to copy a defined 3D geometry object into the fit 3D metrology context.

If you perform a fit operation using a depth map image buffer, it is possible to specify a ROI in raster format, where only the values inside the ROI will be used in the fit. Note that missing data (or invalid points) in the depth map (or point cloud) are ignored when fitting.

When fitting, it is also possible to specify an outlier distance, beyond which no input data is considered for the fit operation.

After calling **M3dmetFit**(), you can inquire whether the operation was successful using M3dmetGetResult() with M_STATUS_FIT, along with several other metrics. To retrieve, from a fit 3D metrology result buffer, the calculated best fit 3D geometry or the inliers, use M3dmetCopyResult().

If statistics about the fit are not required, you can specify a 3D geometry object in which to directly copy the fit result. In this case, if the fit succeeds, **M3dmetFit**() defines the 3D geometry with the specified type.

> Note that 3D geometries of type M_BOX and M_POINT are not supported for this function.

Parameters

Context3dmetId

Specifies the fit 3D metrology context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 3D metrology fit context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to perform the fit operation using all the default settings specified for fit 3D metrology contexts in M3dmetControl().	c		e	f	g	a		i		j	k	l	m	n	b
Fit 3D metrology context identifier	Specifies the identifier of a fit 3D metrology context. This context must have been previously allocated on the required system using M3dmetAlloc() with M_FIT_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerOrImageBufId

Specifies the point cloud or depth map to which to fit the 3D geometry.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud or depth map

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Depth map container identifier	Specifies the identifier of the container containing a 3D-processable depth map. The container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE).	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of an image buffer that contains a fully corrected depth map. The image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned buffer and must be fully corrected (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). This image buffer can have a region of interest (ROI) associated with it, in raster format. Only values inside the ROI will be used in the fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies the identifier of the container containing a 3D-processable point cloud. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE).	c		e	f	g	a		i		j	k	l	m	n	b

GeometryType

Specifies the type of 3D geometry object to use in the fit operation. Note that 3D box and 3D point geometry objects are not supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Specifies the type of geometry to fit onto the source container																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CYLINDER	Specifies to fit a 3D cylinder geometry to the point cloud or depth map. Fitting a 3D cylinder geometry requires at least 5 valid data points. > Note that the fit operation considers the 3D cylinder geometry's curved surface only. If the target cylinder in the point cloud or depth map has bases, then the operation might try to fit the 3D cylinder geometry to the data points on the bases, giving inaccurate results. You can set an OutlierDistance to include only data points that are found within the specified distance from the 3D cylinder geometry's curved surface, preventing an erroneous fit to data points that make up a cylinder's base in the point cloud or depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE	Specifies to fit a 3D line geometry to the point cloud or depth map. Fitting a 3D line geometry requires at least 2 valid data points.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANE	Specifies to fit a 3D plane geometry to the point cloud or depth map. Fitting a 3D plane geometry requires at least 3 valid data points.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPHERE	Specifies to fit a 3D sphere geometry to the point cloud or depth map. Fitting a 3D sphere geometry requires at least 4 valid data points.	c		e	f	g	a		i		j	k	l	m	n	b

Result3dmetOr3dgeoId

Specifies the identifier of the fit 3D metrology result buffer in which to store the results of the fit operation, or specifies the 3D geometry object in which to copy a successful fit result.

The result buffer must have been previously allocated using M3dmetAllocResult() with M_FIT_RESULT. To retrieve statistics for the fit operation, use M3dmetGetResult().

If you specify a 3D geometry object, a successful fit operation results in a defined 3D geometry of the specified type. Note that statistics results are not available if choosing this option. If the fit fails, the geometry's type will return M_NOT_INITIALIZED (if inquired using M3dgeoInquire() with M_GEOMETRY_TYPE).

OutlierDistance

Specifies the distance from the 3D geometry, beyond which points are considered outliers and will be ignored during the fit operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the distance beyond which points become outliers

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies to automatically compute the outlier distance. You can retrieve the calculated value using M3dmetGetResult() with M_OUTLIER_DISTANCE. When specified, M_AUTO_VALUE uses the expected percentage of outliers (M3dmetControl() with M_EXPECTED_OUTLIER_PERCENTAGE) in the outlier distance computation.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that no points in the point cloud are outliers, and all points will be considered inliers in every iteration of the fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the distance from the 3D geometry, beyond which points are considered outliers and will be ignored during the fit operation.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetFree

Synopsis

Free a 3D metrology context or result buffer.

Syntax

```
void M3dmetFree(
    AIL_ID ContextOrResult3dmetId //in
)
```

Description

This function deletes the specified 3D metrology context or result buffer identifier, and release any memory associated with it.

All 3D metrology contexts or 3D metrology result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResult3dmetId

Specifies the identifier of the 3D metrology context or result buffer to free. These must have been successfully allocated using M3dmetAlloc() or M3dmetAllocResult() prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetGetResult

Synopsis

Get the specified type of result(s) from a 3D metrology result buffer.

Syntax

```
AIL_DOUBLE M3dmetGetResult(
    AIL_ID Result3dmetId, //in
    AIL_INT64 ResultType, //in
    void *ResultArrayPtr  //out
)
```

Description

This function retrieves the result(s) of the specified type from a 3D metrology result buffer. For calculate 3D metrology result buffers, results are available after calling M3dmetDistanceEx(), M3dmetFeatureEx(), or M3dmetVolumeEx(). For fit 3D metrology result buffers, results are available after calling M3dmetFit(). For statistics 3D metrology result buffers, results are available after calling M3dmetStat().

Parameters

Result3dmetId

Specifies the identifier of a calculate 3D metrology result buffer, fit 3D metrology result buffer, or statistics 3D metrology result buffer, from which to retrieve results.

See the Parameter associations section for possible values that can be specified.

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the results. Since the **M3dmetGetResult**() function also returns the results, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **Result3dmetId** and **ResultType** parameters and possible values returned to the **ResultArrayPtr** parameter.

- For specifying the type of result to retrieve from a calculate, fit, or statistics 3D metrology result buffer

The following Result3dmetId, ResultType, and ResultArrayPtr parameter settings can be specified for different types of 3D metrology result buffers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result to retrieve from a calculate, fit, or statistics 3D metrology result buffer																	
Result3dmetId ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Calculate 3D metrology result buffer ID, for retrieving results of an M3dmetDistanceEx operation	Specifies the identifier of a calculate 3D metrology result buffer that holds results of an M3dmetDistanceEx() operation.															

M_RESULT_IMAGE_SIZE_X +	Retrieves the size in X of the source point cloud's range component or of the source depth map, in pixels. This is the same size in X as the source depth map or as the range of the source container used in the M3dmetDistanceEx() operation. Note, M_STATUS_DISTANCE must not be M_NOT_INITIALIZED.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_IMAGE_SIZE_Y +	Retrieves the size in Y of the source point cloud's range component or of the source depth map, in pixels. This is the same size in Y as the source depth map or as the range of the source container used in the M3dmetDistanceEx() operation. Note, M_STATUS_DISTANCE must not be M_NOT_INITIALIZED.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_DISTANCE +	Retrieves the global status of the last M3dmetDistanceEx().	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies that M3dmetDistanceEx() was called and the operation has been completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_INITIALIZED	Specifies that the distance 3D metrology result buffer was not use in a call to M3dmetDistanceEx() and contains no results.	c		e	f	g	a		i		j	k	l	m	n	b
Calculate 3D metrology result buffer ID, for retrieving results of an M3dmetFeatureEx operation	Specifies the identifier of a calculate 3D metrology result buffer that holds results of an M3dmetFeatureEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_ALONG_LINE(AIL_INT N) +	Retrieves the distance of the specified point along the source 3D line geometry object. This result is available after calling M3dmetFeatureEx() with M_INTERSECTION or M_RAY_CAST if one of the source geometries used was of type M_LINE.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	N Specifies the index of the point on the line.															
	0 <= *Value* <*M3dmetGetResult*(M_NUMBER) — Specifies the valid index of the point.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE +	Retrieves the angle between source geometries. This result is available after calling M3dmetFeatureEx() with M_ANGLE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_TO_EDGE_CASE +	Retrieves the angle of the deviation from a degenerate case. This result is available after calling M3dmetFeatureEx() with M_ORTHOGONALIZE, M_PROJECTION, M_RAY_CAST, or M_INTERSECTION when not calculating the intersection points between a line and a geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CALCULATED_OBJECT_TYPE +	Retrieves the type of object contained in the specified calculate 3D metrology result buffer. This result is always available.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_GEOMETRY	Specifies that a 3D geometry object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies that there is no calculated object contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFORMATION_MATRIX	Specifies that a transformation matrix object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIFFERENCE +	Retrieves the difference between the union and the intersection of two source geometry objects. This result is available after calling M3dmetFeatureEx() with M_OVERLAP.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE +	Retrieves the distance calculated during the M3dmetFeatureEx() operation. This result is available after calling M3dmetFeatureEx() with M_DISTANCE, M_SHORTEST_LINE, M_CLOSEST_POINT, or M_FARTHEST_POINT. Note that M_DISTANCE can return either the shortest or the longest distance, depending on the operation used.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_GEOMETRY_TYPE +	Retrieves the type of geometry contained in the specified calculate 3D metrology result buffer. This result is available when M_CALCULATED_OBJECT_TYPE returns M_GEOMETRY.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX	Specifies a box geometry object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CYLINDER	Specifies a cylinder geometry object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE	Specifies a line geometry object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies that there is no geometry contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANE	Specifies a plane geometry object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINT	Specifies a point geometry object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPHERE	Specifies a sphere geometry object is contained within the specified calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERSECTION +	Retrieves the volume of the intersection between two geometries. This result is available after calling M3dmetFeatureEx() with M_OVERLAP.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_IS_INSIDE +	Retrieves whether the first specified source geometry object is inside of the second source geometry object. This result is available after calling M3dmetFeatureEx() with M_CLIP or M_IS_INSIDE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INSIDE	Specifies that the first source geometry object is completely contained within the second source geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTSIDE	Specifies that the first source geometry object is not contained within the second source geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARTIALLY_INSIDE	Specifies that the first source geometry object is partially contained within the second source geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +	Retrieves the number of geometries in the specified calculate 3D metrology result buffer. This result is always available.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_OPERATION +	Retrieves the operation used during the last call to M3dmetFeatureEx(). This result is always available.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE	Specifies that the angle between source 3D geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BOUNDING_BOX	Specifies that the axis-aligned bounding box of source 3D geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CLIP	Specifies that a 3D line geometry object was truncated such that it lies inside a second 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_POINT	Specifies that the point on the second specified 3D geometry that is closest to the first specified 3D geometry was calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE	Specifies that the distance between source 3D geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRUSION_BORDER	Specifies that an extrusion of a 3D box's face was calculated such that the box's closest corner meets the specified plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRUSION_CENTER	Specifies that an extrusion of a 3D box's face was calculated such that the center of the face touches the specified plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_FARTHEST_POINT	Specifies that the point on the second specified 3D geometry that is farthest from the first specified 3D geometry was calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION	Specifies that the interpolation between source 3D geometries or matrices was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERSECTION	Specifies that the intersection line, points, or polygon between source 3D geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_INSIDE	Specifies that whether the first source 3D geometry is inside, outside, or partially inside a second 3D geometry was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_NORMAL_AT_POSITION	Specifies that a unit normal line (one unit in length) was calculated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_INITIALIZED	Specifies that M3dmetFeatureEx() has not been called with this calculate 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ORTHOGONALIZE	Specifies that the specified source 3D geometry was rotated such that it is arranged orthogonally with another 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OVERLAP	Specifies that the percentage of volume overlap between source 3D geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PARALLELISM	Specifies that the angular deviation from a parallel orientation for source 3D geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PERPENDICULARITY	Specifies that the angular deviation from a perpendicular orientation for source 3D geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_POINT_ON_LINE	Specifies that a point on the source 3D line was created.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POINT_ON_LINE_CLIPPED	Specifies that a point on the source 3D line was created such that the point does not fall outside the limits of the line.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROJECTION	Specifies that a point or line was projected onto a second 3D geometry, creating a new point or line on the surface of the second 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RAY_CAST	Specifies that the intersection point(s) between a ray and a second source 3D geometry, where a ray is a line that extends infinitely in one direction from the line's start point was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SHORTEST_LINE	Specifies that the shortest line between two source 3D line geometries was calculated. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERLAP +		Retrieves the percentage of overlap between two source 3D geometries. This result is available after calling M3dmetFeatureEx() with M_OVERLAP.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PARALLELISM +		Retrieves the deviation from a parallel orientation between two source 3D geometries. This result is available after calling M3dmetFeatureEx() with M_ANGLE or M_PARALLELISM.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PERPENDICULARITY +		Retrieves the deviation from a perpendicular orientation between two source 3D geometries. This result is available after calling M3dmetFeatureEx() with M_ANGLE or M_PERPENDICULARITY.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_EXTRUSION +		Retrieves whether the extrusion operation was successful. This result is available after calling M3dmetFeatureEx() with M_EXTRUSION_CENTER or M_EXTRUSION_BORDER.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_FAIL	Specifies that the extrusion operation did not complete successfully. Failed extrusions can occur for an M_EXTRUSION_BORDER operation when the plane intersects opposing parallel faces of the box.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the extrusion operation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNION +	Retrieves the volume of the union of two source 3D geometries. This result is available after calling M3dmetFeatureEx() with M_OVERLAP.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Calculate 3D metrology result buffer ID, for retrieving results of an M3dmetVolumeEx operation	Specifies the identifier of a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_ELEMENT_IMAGE_SIZE_X +	Retrieves the size in X of the source point cloud's mesh component or of the source depth map, in pixels. You can use this value to allocate an image buffer of the correct size when retrieving a result image created using M3dmetCopyResult() with M_VOLUME_ELEMENT_INDEX_IMAGE, M_VOLUME_ELEMENT_MASK, or M_VOLUME_ELEMENT_STATUS_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_ELEMENT_IMAGE_SIZE_Y +	Retrieves the size in Y of the source point cloud's mesh component or of the source depth map, in pixels. You can use this value to allocate an image buffer of the correct size when retrieving a result image created using M3dmetCopyResult() with M_VOLUME_ELEMENT_INDEX_IMAGE, M_VOLUME_ELEMENT_MASK, or M_VOLUME_ELEMENT_STATUS_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_IMAGE_SIZE_X +	Retrieves the size in X of the source point cloud's range component or of the source depth map, in pixels. You can use this value to allocate an image buffer of the correct size when retrieving a result image created using M3dmetCopyResult() with M_VOLUME_SOURCE_POINTS_MASK or M_VOLUME_SOURCE_POINTS_STATUS_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_IMAGE_SIZE_Y +	Retrieves the size in Y of the source point cloud's range component or of the source depth map, in pixels. You can use this value to allocate an image buffer of the correct size when retrieving a result image created using M3dmetCopyResult() with M_VOLUME_SOURCE_POINTS_MASK or M_VOLUME_SOURCE_POINTS_STATUS_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_VOLUME_INFO +	Retrieves whether volume information has been saved into the calculate 3D metrology result buffer. Volume information is necessary to copy results into an image buffer or container (for example, to create a status image using M3dmetCopyResult() with M_VOLUME_ELEMENT_STATUS_IMAGE). Volume information is only saved if M3dmetControl() with M_SAVE_VOLUME_INFO is set to M_TRUE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that volume information has not been saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that volume information has been saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_VOLUME +	Retrieves the status of the volume operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_FAIL_GAPS	Specifies that the volume was not successfully computed, since the source Aurora Imaging Library object was a mesh with gaps, and there was no reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAIL_INVALID_MESH	Specifies that the volume was not successfully computed, since the mesh was not usable.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_INITIALIZED	Specifies that the calculate 3D metrology result buffer was not used in a call to M3dmetVolumeEx(), and contains no results.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SUCCESS	Specifies that the volume was successfully computed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WARNING_GAPS	Specifies that the volume was successfully computed, but there were gaps in the specified source depth map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME +		Retrieves the computed volume.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_MODE +		Retrieves the volume mode used to calculate the volume.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABOVE	Specifies that the total volume is equal only to the volume above the reference Aurora Imaging Library object. Any volume below the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPLETE	Specifies to calculate all volume results, which can then be retrieved, copied, or drawn, according to the specified M_VOLUME_OUTPUT_MODE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIFFERENCE	Specifies that the total volume is calculated as the volume above the reference Aurora Imaging Library object, minus the volume below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TOTAL	Specifies that the total volume is calculated as the sum of all volumes between the source and reference Aurora Imaging Library objects, whether above or below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNDER	Specifies that the total volume is equal only to the volume under the reference Aurora Imaging Library object. Any volume above the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_NB_ELEMENTS +		Retrieves the number of elements that were used to compute the volume.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_NB_NEGATIVE_ELEMENTS +		Retrieves the number of elements that were used to compute the negative part of the volume. For more information, see the How the volume of a meshed point cloud with holes is calculated subsection of the Calculating a volume section of Chapter 39: 3D metrology.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_NB_POSITIVE_ELEMENTS +		Retrieves the number of elements that were used to compute the positive part of the volume.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_VOLUME_NB_UNUSED_ELEMENTS +	Retrieves the number of valid elements that were not used to compute the volume. To illustrate, for a 10 x 10 depth map with 5 pixels corresponding to invalid data, 95 valid elements remain for a possible volume calculation. If the specified operation is M_ABOVE and 40 elements contribute positively to the volume calculation, while 0 contribute negatively, then there are 55 valid elements that were not used to calculate the volume (95 - 40 - 0 = 55), and, for this example, M_VOLUME_NB_UNUSED_ELEMENTS returns 55.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_REFERENCE_TYPE +	Retrieves the type of reference object that was used to compute the volume.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that a reference object was not used.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEPTH_MAP	Specifies a depth map reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANE	Specifies a plane reference object.	c		e	f	g	a		i		j	k	l	m	n	b
Fit 3D metrology result ID	Specifies the identifier of a fit 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_X +	Retrieves the X-component of the fitted 3D geometry's unit vector. If the fitted 3D geometry is a cylinder, the X-component of the cylinder's central axis unit vector is retrieved. This vector does not reflect the cylinder's length. If the fitted 3D geometry is a line, the X-component of the line's direction unit vector is retrieved. This vector does not reflect the line's length.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the X-component of the unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_Y +	Retrieves the Y-component of the fitted 3D geometry's unit vector. If the fitted 3D geometry is a cylinder, the Y-component of the cylinder's central axis unit vector is retrieved. This vector does not reflect the cylinder's length. If the fitted 3D geometry is a line, the Y-component of the line's direction unit vector is retrieved. This vector does not reflect the line's length.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the Y-component of the unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_AXIS_Z +	Retrieves the Z-component of the fitted 3D geometry's unit vector. If the fitted 3D geometry is a cylinder, the Z-component of the cylinder's central axis unit vector is retrieved. This vector does not reflect the cylinder's length. If the fitted 3D geometry is a line, then the Z-component of the line's direction unit vector is retrieved. This vector does not reflect the line's length.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the Z-component of the unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

M_CENTER_X +	Retrieves the X-coordinate of the center point of the fitted 3D geometry, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the X-coordinate of the center point on the cylinder's central axis is retrieved. If the fitted 3D geometry is a line, the X-coordinate of the center point on the line is retrieved. If the fitted 3D geometry is a plane, the X-coordinate of the plane's center is retrieved. This center point is equivalent to the centroid of all inlier points. If the fitted 3D geometry is a sphere, the X-coordinate of the sphere's center point is retrieved.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Y +	Retrieves the Y-coordinate of the center point of the fitted 3D geometry expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the Y-coordinate of the center point on the cylinder's central axis is retrieved. If the fitted 3D geometry is a line, the Y-coordinate of the center point on the line is retrieved. If the fitted 3D geometry is a plane, the Y-coordinate of the plane's center is retrieved. This center point is equivalent to the centroid of all inlier points. If the fitted 3D geometry is a sphere, the Y-coordinate of the sphere's center point is retrieved.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_Z +	Retrieves the Z-coordinate of the center point of the fitted 3D geometry, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the Z-coordinate of the center point on the cylinder's central axis is retrieved. If the fitted 3D geometry is a line, the Z-coordinate of the center point on the line is retrieved. If the fitted 3D geometry is a plane, the Z-coordinate of the plane's center is retrieved. This center point is equivalent to the centroid of all inlier points. If the fitted 3D geometry is a sphere, the Z-coordinate of the sphere's center point is retrieved.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_TO_ORIGIN_X +	Retrieves the X-coordinate of the point on the fitted 3D plane geometry, closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_TO_ORIGIN_Y +	Retrieves the Y-coordinate of the point on the fitted 3D plane geometry, closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_TO_ORIGIN_Z +	Retrieves the Z-coordinate of the point on the fitted 3D plane geometry, closest to the origin of the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COEFFICIENT_A +	Retrieves the coefficient *A* of the fitted 3D plane geometry's equation, Ax + By + Cz + D = 0.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the coefficient *A*.	c		e	f	g	a		i		j	k	l	m	n	b

M_COEFFICIENT_B +	Retrieves the coefficient *B* of the fitted 3D plane geometry's equation, Ax + By + Cz + D = 0.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the coefficient *B*.	c		e	f	g	a		i		j	k	l	m	n	b
M_COEFFICIENT_C +	Retrieves the coefficient *C* of the fitted 3D plane geometry's equation, Ax + By + Cz + D = 0.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the coefficient *C*.	c		e	f	g	a		i		j	k	l	m	n	b
M_COEFFICIENT_D +	Retrieves the coefficient *D* of the fitted 3D plane geometry's equation, Ax + By + Cz + D = 0.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the coefficient *D*.	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_X +	Retrieves the X-coordinate of the end point of the fitted 3D geometry, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the X-coordinate of the cylinder's end point (positioned at the center of the cylinder's second circular base) is retrieved. If the fitted 3D geometry is a line, the X-coordinate of the line's end point is retrieved.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Y +	Retrieves the Y-coordinate of the end point of the fitted 3D geometry, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the Y-coordinate of the cylinder's end point (positioned at the center of the cylinder's second circular base) is retrieved. If the fitted 3D geometry is a line, the Y-coordinate of the line's end point is retrieved.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POINT_Z +	Retrieves the Z-coordinate of the end point of the fitted 3D geometry, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the Z-coordinate of the cylinder's end point (positioned at the center of the cylinder's second circular base) is retrieved. If the fitted 3D geometry is a line, the Z-coordinate of the line's end point is retrieved.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_RMS_ERROR +	Retrieves the root-mean-squared (RMS) error of the distance between the point cloud or depth map, and the fitted 3D geometry. Only inliers are considered when calculating the RMS error. For planes, the error corresponds to the distance between the points and the fitted plane. For spheres and cylinders, the error corresponds to the distance between the points and the surface of the sphere or cylinder. For cylinders, this specifically refers to the distance to the curved surface and not the circular bases. This result is only available for retrieval after a successful fit.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_GEOMETRY_TYPE +	Retrieves the type of the fitted 3D geometry. This result is only available for retrieval after a successful fit.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CYLINDER	Specifies a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINE	Specifies a line.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANE	Specifies a plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPHERE	Specifies a sphere.	c		e	f	g	a		i		j	k	l	m	n	b
M_LENGTH +	Retrieves the length of the fitted 3D cylinder geometry, or fitted 3D line geometry, in world units.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL_X +	Retrieves the X-component of the fitted 3D plane geometry's normal unit vector.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the X-component of the normal unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL_Y +	Retrieves the Y-component of the fitted 3D plane geometry's normal unit vector.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the Y-component of the normal unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL_Z +	Retrieves the Z-component of the fitted 3D plane geometry's normal unit vector.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
-1.0 >= *Value* >= 1.0	Specifies the Z-component of the normal unit vector, expressed in the working coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_INLIERS +	Retrieves the number of points that were considered inliers during the fit operation. This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_MISSING_DATA +	Retrieves the number of points with zero confidence (if you specified a container), or the number of points with an invalid value (if you specified a depth map). Note that in the case of depth maps, this only refers to the points inside the ROI. This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_OUTLIERS +	Retrieves the number of points that were considered outliers during the fit operation. This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_POINTS_TOTAL +	Retrieves the total number of points. This number is equal to M_NUMBER_OF_POINTS_VALID + M_NUMBER_OF_POINTS_MISSING_DATA. This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_POINTS_VALID +	Retrieves the number of valid points, which is equal to the number of points with non-zero confidence. This number is equal to M_NUMBER_OF_POINTS_INLIERS + M_NUMBER_OF_POINTS_OUTLIERS. This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_DISTANCE +	Retrieves the outlier distance used for the fit. When M_AUTO_VALUE was specified, the function returns the automatically calculated value; otherwise, it returns the value passed to M3dmetFit(). This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_RADIUS +	Retrieves the radius of the fitted 3D cylinder geometry or 3D sphere geometry in world units.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_IMAGE_SIZE_X +	Retrieves the size in X of the range component of the source point cloud or depth map, in pixels. Note that you can use this value to allocate an image buffer of an appropriate size for M3dmetCopyResult(). This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size in X.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_IMAGE_SIZE_Y +	Retrieves the size in Y of the range component of the source point cloud or depth map, in pixels. Note that you can use this value to allocate an image buffer of an appropriate size for M3dmetCopyResult(). This result is only available for retrieval after a fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size in Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_START_POINT_X +	Retrieves the X-coordinate of the fitted 3D geometry's start point, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the start point is positioned at the center of the cylinder's first circular base. If the fitted 3D geometry is a line, the start point is positioned at the start of the line.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_START_POINT_Y +	Retrieves the Y-coordinate of the fitted 3D geometry's start point, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the start point is positioned at the center of the cylinder's first circular base. If the fitted 3D geometry is a line, the start point is positioned at the start of the line.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_START_POINT_Z +	Retrieves the Z-coordinate of the fitted 3D geometry's start point, expressed in the working coordinate system. If the fitted 3D geometry is a cylinder, the start point is positioned at the center of the cylinder's first circular base. If the fitted 3D geometry is a line, the start point is positioned at the start of the line.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_FIT +	Retrieves the status of the fit operation. This result is always available for retrieval.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_POINTS_COLLINEAR	Specifies that the fit failed because the fit operation tried to fit a sphere, cylinder, or plane, to points that lie on the same straight line.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_POINTS_COPLANAR	Specifies that the fit failed because the fit operation tried to fit a sphere or cylinder, to points that lie on the same plane.	c		e	f	g	a		i		j	k	l	m	n	b
M_BAD_ESTIMATE	Specifies that the fit failed because the initial fit estimate did not include any valid inliers.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_ENOUGH_VALID_DATA	Specifies that the fit failed because there were not enough valid points to fit the specified 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_INITIALIZED	Specifies that the fit 3D metrology result buffer was not used in a call to M3dmetFit(), and contains no results.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the fit operation completed successfully.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics 3D metrology result ID	Specifies the identifier of a statistics 3D metrology result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX +	Retrieves the maximum distance between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS +	Retrieves the maximum absolute distance between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN +	Retrieves the average distance between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN_ABS +	Retrieves the average absolute distance between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN +	Retrieves the minimum distance between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS +	Retrieves the minimum absolute distance between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_NUMBER +	Retrieves the number of points that satisfied the condition specified when M3dmetStat() was called (using the Condition parameter).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_RMS +	Retrieves the root-mean-square (RMS) error between the point cloud or depth map, and the reference 3D geometry object. Aurora Imaging Library calculates the RMS error using the following formula: $\text{RMS error} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}[(\bar{x}_i - x_i)^2 + (\bar{y}_i - y_i)^2 + (\bar{z}_i - z_i)^2]}$	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION +	Retrieves the standard deviation of all the distances calculated between the point cloud or depth map, and the reference 3D geometry. Aurora Imaging Library calculates the standard deviation using the following formula: $s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}[(\bar{x}_i - x_i)^2 + (\bar{y}_i - y_i)^2 + (\bar{z}_i - z_i)^2]}$	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM +	Retrieves the sum of all the distances calculated between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS +	Retrieves the sum of all the absolute distances calculated between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES +	Retrieves the sum of squared distances between the point cloud or depth map, and the reference 3D geometry.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values of the ResultType parameter listed in For specifying the type of result to retrieve from a calculate, fit, or statistics 3D metrology result buffer.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the ResultType parameter listed in For specifying the type of result to retrieve from a calculate, fit, or statistics 3D metrology result buffer.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type

ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_DOUBLE*. If the requested information does not fit into an *AIL_DOUBLE*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetInquire

Synopsis

Inquire information about a 3D metrology context.

Syntax

```
AIL_INT64 M3dmetInquire(
    AIL_ID ContextOrResult3dmetId, //in
    AIL_INT64 InquireType,         //in
    void *UserVarPtr               //out
)
```

Description

This function inquires information about a 3D metrology context.

To retrieve results from a 3D metrology result buffer, use M3dmetGetResult().

To inquire about draw 3D metrology settings for M3dmetDraw3d(), use M3dmetInquireDraw() instead.

Parameters

ContextOrResult3dmetId

Specifies the identifier of the 3D metrology context or result buffer to inquire. The 3D metrology context or result buffer must have been previously allocated on the required system using M3dmetAlloc() or M3dmetAllocResult(), respectively.

InquireType

Specifies the type of information about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the variable in which to write the requested information. Since the **M3dmetInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a statistics 3D metrology context
- For inquiring about a fit 3D metrology context
- For inquiring about a volume 3D metrology context
- For inquiring about a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx operation
- For inquiring about a distance 3D metrology context

For a statistics 3D metrology context, the InquireType parameter can be set to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a statistics 3D metrology context																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STAT_MAX +	Inquires whether the maximum distance between the point cloud or depth map, and the specified reference object will be calculated. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies not to calculate the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS +		Inquires whether the maximum absolute distance between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the maximum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the maximum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN +		Inquires whether the average distance between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the average distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the average distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN_ABS +		Inquires whether to calculate the average absolute distance between the point cloud or depth map, and the specified reference object. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the average absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the average absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN +		Inquires whether the minimum distance between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the minimum distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the minimum distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS +		Inquires whether the minimum absolute distance between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to calculate the minimum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to calculate the minimum absolute distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_NUMBER +		Inquires whether to record the number of points that meet the condition specified when calling M3dmetStat() (using the Condition, CondLow and CondHigh parameters). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to record the number of points that meet the specified condition.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to record the number of points that meet the specified condition.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_RMS +	Inquires whether the root-mean-square (RMS) error between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the RMS error.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the RMS error.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION +	Inquires whether the standard deviation of all the distances calculated between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the standard deviation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM +	Inquires whether the sum of all the distances calculated between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the sum of all the distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the sum of all the distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS +	Inquires whether the sum of all the absolute distances calculated between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the sum of all absolute distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the sum of all absolute distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES +	Inquires whether the sum of squared distances between the point cloud or depth map, and the specified reference object will be calculated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to calculate the sum of squared distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to calculate the sum of squared distances.	c		e	f	g	a		i		j	k	l	m	n	b

For a fit 3D metrology context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a fit 3D metrology context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ESTIMATION_MODE +	Inquires how to compute an initial fit estimate between the point cloud or depth map, and the specified reference object. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_FROM_GEOMETRY	Specifies that an initial fit estimate is determined using a geometry object.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_SAMPLING	Specifies that an initial fit estimate is calculated using all available points in a point cloud or depth map.	c		e	f	g	a		i		j	k	l	m	n	b
M_RANDOM_SAMPLING	Specifies that an initial fit estimate is determined using a random sampling consensus (RANSAC) algorithm.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXPECTED_OUTLIER_PERCENTAGE +	Inquires the expected percentage of outliers among the points of the point cloud or depth map to be fitted. Note, this is only used for M3dmetFit() operations with OutlierDistance set to M_AUTO_VALUE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* < 100.0	Specifies the expected percentage of outliers among the points of the point cloud or depth map to be fitted.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_ITERATIONS_MAX +	Inquires the maximum number of iterations to use during the fit operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations to use during the fit operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_INLIER_AMOUNT_THRESHOLD +	Inquires the minimum number of inliers required for the fit operation to end. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that there is no minimum number of inliers.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the minimum number of inliers.	c		e	f	g	a		i		j	k	l	m	n	b
M_RMS_ERROR_THRESHOLD +	Inquires the maximum RMS error required for the fit operation to end. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum RMS error.	c		e	f	g	a		i		j	k	l	m	n	b

For a volume 3D metrology context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a volume 3D metrology context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SAVE_VOLUME_INFO +	Inquires whether to save information from the volume computation for use with M3dmetCopyResult() or M3dmetDraw3d() operations. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the volume information is not saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the volume information is saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_MODE +	Inquires how to calculate the total volume, given the position of the source Aurora Imaging Library object relative to the reference object. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABOVE	Specifies that the total volume is equal only to the volume above the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPLETE	Specifies to calculate all volume results, which can then be retrieved, copied, or drawn, according to the specified M_VOLUME_OUTPUT_MODE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIFFERENCE	Specifies that the total volume is calculated as the volume above the reference Aurora Imaging Library object, minus the volume below the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TOTAL	Specifies that the total volume is calculated as the sum of all volumes between the source and reference Aurora Imaging Library objects, whether above or below the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNDER	Specifies that the total volume is equal only to the volume under the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx() operation, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx operation																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_VOLUME_OUTPUT_MODE +		Inquires the volume mode for which to retrieve, copy, or draw results, using M3dmetGetResult(), M3dmetCopyResult(), or M3dmetDraw3d(), respectively, when M_VOLUME_MODE is set to M_COMPLETE. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABOVE	Specifies that the total volume is equal only to the volume above the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to match the M_VOLUME_MODE setting, unless M_VOLUME_MODE is set to M_COMPLETE, in which case M_TOTAL is used.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIFFERENCE	Specifies that the total volume is calculated as the volume above the reference Aurora Imaging Library object, minus the volume below the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TOTAL	Specifies that the total volume is calculated as the sum of all volumes between the source and reference Aurora Imaging Library objects, whether above or below the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNDER	Specifies that the total volume is equal only to the volume under the reference Aurora Imaging Library object. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a distance 3D metrology context, the InquireType parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a distance 3D metrology context																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DISTANCE_METRIC_OPERATOR +		Inquires the operator applied to the distance from M_PAIRING_DISTANCE_METRIC. SET															

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE	Specifies to use an absolute metric when calculating the distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED	Specifies to use a signed metric when calculating the distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SQUARED	Specifies to use a squared metric when calculating the distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_NORMAL +		Inquires whether to compute the normal distance as the dot product between the paired source point's normal and the reference point's normal. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not compute the normal distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to compute the normal distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRING_DISTANCE_MAX +		Inquires the maximum distance between a source point and a reference point to be paired. Distances larger than this are invalid. Note, this is only used when the reference object is a depth map or a point cloud. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies no maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRING_DISTANCE_METRIC +		Inquires the metric used when choosing a source point's paired reference element. The paired reference element is that which is closest to a source point using the specified metric. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to automatically choose the metric based on the reference object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIRECTION_NORMAL	Specifies to use the distance in the normal direction of the source point. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DIRECTION_Z	Specifies to use the distance in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EUCLIDEAN	Specifies to use the Euclidean distance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MANHATTAN	Specifies to use the Manhattan distance.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRING_PERPENDICULAR_DISTANCE_MAX +		Inquires the maximum perpendicular distance between a source point and a reference point to be paired when using M_DISTANCE_TO_NEIGHBOR. Distances greater than this are invalid. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum perpendicular distance.	c		e	f	g	a		i		j	k	l	m	n	b

M_PAIRING_PERPENDICULAR_DISTANCE_MAX_MODE +		Inquires whether an automatic value or a user-defined value is used for the maximum perpendicular distance between a source point and a reference point to be paired. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies an automatic value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies to use the user-defined value specified using M_PAIRING_PERPENDICULAR_DISTANCE_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
M_PAIRING_TYPE +		Inquires against which part of the reference object to measure when performing an M3dmetDistanceEx() operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	M_AUTO is the same as the first available setting: M_DISTANCE_TO_SURFACE, M_DISTANCE_TO_NEIGHBOR, M_DISTANCE_TO_REFERENCE in decreasing preference.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_NEIGHBOR	Specifies to calculate the distance between the points of the source point cloud or depth map, and their nearest point in the specified reference point cloud or depth map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_REFERENCE	Specifies to calculate the distance between the points of the source point cloud or depth map and the specified reference 3D geometry. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_SURFACE	Specifies to calculate the distance between the points of the source point cloud or depth map, and the surface of the specified reference object. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_PAIRED_REFERENCE_FACET +		Inquires whether to save the index of the reference facet (face) to each point. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not save the paired reference facet.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save the paired reference facet.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_PAIRED_REFERENCE_PIXEL +		Inquires whether to save the index of the paired reference pixel to each source point. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not save the paired reference pixel.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save the paired reference pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_PAIRED_REFERENCE_POINT +		Inquires whether to save the coordinates of the paired reference point to each source point. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to not save the coordinates of the paired reference points.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save the coordinates of the paired reference points.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for

Tables:

- For inquiring about a statistics 3D metrology context
- For inquiring about a fit 3D metrology context
- For inquiring about a volume 3D metrology context
- For inquiring about a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx operation
- For inquiring about a distance 3D metrology context

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to get information about the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the default value																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DEFAULT	Inquires the default value of the specified inquire type.															
UserVarPtr extra info	Data type info: Data type: *defined elsewhere*	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_SET_TO_DEFAULT	Inquires whether the specified inquire type is set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For inquiring about a statistics 3D metrology context
- For inquiring about a fit 3D metrology context
- For inquiring about a volume 3D metrology context
- For inquiring about a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx operation
- For inquiring about a distance 3D metrology context

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire whether an inquire type is supported																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_HAS_DEFAULT	Inquires whether the specified inquire type has a default value.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED	Inquires whether the specified inquire type is supported for the 3D metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for

Tables:

- For inquiring about a statistics 3D metrology context
- For inquiring about a fit 3D metrology context
- For inquiring about a volume 3D metrology context
- For inquiring about a calculate 3D metrology result buffer that holds results of an M3dmetVolumeEx operation
- For inquiring about a distance 3D metrology context

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to a required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To specify the data type																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	
M_TYPE_AIL_DOUBLE		Casts the requested information to an *AIL_DOUBLE*.																
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT		Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT		Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32		Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, if it is an *AIL_INT*. If the requested information is an *AIL_DOUBLE* or if it is an *AIL_INT64* on a 32-bit system, this function will return **M_NULL**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetInquireDraw

Synopsis

Inquire about a setting of a draw 3D metrology context.

Syntax

```
AIL_INT64 M3dmetInquireDraw(
    AIL_ID DrawContext3dmetId, //in
    AIL_INT64 Operation,       //in
    AIL_INT64 InquireType,     //in
    void *UserVarPtr           //out
)
```

Description

This function inquires about a specified setting of a draw 3D metrology context. These settings establish which metrology results to draw into the 3D graphics list when calling M3dmetDraw3d() and how to draw them.

You can control the draw 3D metrology context settings using M3dmetControlDraw().

Parameters

DrawContext3dmetId

Specifies the identifier of the draw 3D metrology context to inquire. The draw 3D metrology context must have been previously allocated on the required system using M3dmetAlloc() with M_DRAW_3D_CONTEXT.

Operation

Specifies the draw operation about which to inquire.

See the Parameter associations section for possible values that can be specified.

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **M3dmetInquireDraw**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **Operation** and **InquireType** parameters and possible values returned to the **UserVarPtr** parameter.

- For inquiring a draw 3D metrology context

The following Operation and InquireType parameter settings are available to inquire about draw 3D metrology contexts settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring a draw 3D metrology context																		
Operation			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	InquireType																	
		UserVarPtr - Possible values returned																

M_DRAW_VOLUME_ELEMENTS	Inquires whether and how to draw the volume elements used to compute the volume, independent of whether the elements contributed positively or negatively to the volume.														
M_ACTIVE +	Inquires whether to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_APPEARANCE +	Inquires whether to draw the graphic(s) as a solid surface, wireframe, or points. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINTS	Specifies a points appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID	Specifies a solid appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_POINTS	Specifies a solid appearance with points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_WIREFRAME	Specifies a solid appearance with a wireframe. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WIREFRAME	Specifies a wireframe appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR +	Inquires the color with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_GREEN	Specifies the color green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPACITY +		Inquires the opacity with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the opacity of the 3D graphic. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKNESS +		Inquires the thickness with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the point/line thickness, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_ELEMENT_APPEARANCE +		Inquires whether to draw the volume element(s) as a surface or volume. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE	Specifies to draw the surfaces of the source and reference objects that were specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE_REFERENCE	Specifies to draw only the surfaces of the reference object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE_SOURCE	Specifies to draw only the surfaces of the source object that was specified for the M3dmetVolumeEx() operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VOLUME	Specifies to draw each volume element as a distinguishable volume. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_VOLUME_NEGATIVE_ELEMENTS	Inquires whether and how to draw the volume elements that contributed negatively to the volume computation; these elements decreased the volume.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACTIVE +	Inquires whether to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the graphic(s). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_APPEARANCE +	Inquires whether to draw the graphic(s) as a solid surface, wireframe, or points. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINTS	Specifies a points appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID	Specifies a solid appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_POINTS	Specifies a solid appearance with points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_WIREFRAME	Specifies a solid appearance with a wireframe. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WIREFRAME	Specifies a wireframe appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR +	Inquires the color with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_GREEN	Specifies the color green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPACITY +		Inquires the opacity with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the opacity of the 3D graphic. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKNESS +		Inquires the thickness with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the point/line thickness, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_ELEMENT_APPEARANCE +		Inquires whether to draw the volume element(s) as a surface or volume. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE	Specifies to draw the surfaces of the source and reference objects that were specified for the M3dmetVolumeEx() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE_REFERENCE	Specifies to draw only the surfaces of the reference object that was specified for the M3dmetVolumeEx() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE_SOURCE	Specifies to draw only the surfaces of the source object that was specified for the M3dmetVolumeEx() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_VOLUME	Specifies to draw each volume element as a distinguishable volume. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_VOLUME_POSITIVE_ELEMENTS	Inquires wwhether and how to draw the volume elements that contributed positively to the volume computation; these elements increased the volume.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACTIVE +	inquires whether to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to draw the graphic(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to draw the graphic(s). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_APPEARANCE +	Inquires whether to draw the graphic(s) as a solid surface, wireframe, or points. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
M_POINTS	Specifies a points appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID	Specifies a solid appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_POINTS	Specifies a solid appearance with points. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SOLID_WITH_WIREFRAME	Specifies a solid appearance with a wireframe. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WIREFRAME	Specifies a wireframe appearance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR +	Inquires the color with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED	Specifies the color dark red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW	Specifies the color dark yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY	Specifies the color gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_LIGHT_BLUE	Specifies the color light blue. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPACITY +		Inquires the opacity with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the opacity of the 3D graphic. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THICKNESS +		inquires the thickness with which to draw the graphic(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_GRAPHIC_LIST_DEFAULT	Specifies to use the 3D graphics list's default setting.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the point/line thickness, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_ELEMENT_APPEARANCE +		Inquires whether to draw the volume element(s) as a surface or volume. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE	Specifies to draw the surfaces of the source and reference objects that were specified for the M3dmetVolumeEx() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE_REFERENCE	Specifies to draw only the surfaces of the reference object that was specified for the M3dmetVolumeEx() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SURFACE_SOURCE	Specifies to draw only the surfaces of the source object that was specified for the M3dmetVolumeEx() operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_VOLUME	Specifies to draw each volume element as a distinguishable volume. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_DRAW_SETTINGS		Inquires a global draw 3D metrology context setting.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VOLUME_ELEMENT_INDEX +	Inquires the index of the volume element to draw. SET	c		e	f	g	a		i		j	k	l	m	n	b

		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_ALL	Specifies to draw all the volume elements.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0	Specifies the index.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the InquireType parameter listed in For inquiring a draw 3D metrology context.

You can add one of the following values to the above-mentioned values to get information about the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the default value																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	
M_DEFAULT		Inquires the default value of the specified inquire type.																
	UserVarPtr extra info	Data type info: Data type: *defined elsewhere*	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_SET_TO_DEFAULT		Inquires whether the specified inquire type is set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is set to its default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the InquireType parameter listed in For inquiring a draw 3D metrology context.

You can add one of the following values to the above-mentioned values to determine whether an inquire type has a default value or whether it is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type has a default value or whether it is supported																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	
M_HAS_DEFAULT		Inquires whether the specified inquire type has a default value.																
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED		Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the InquireType parameter listed in For inquiring a draw 3D metrology context.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																		
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT64*. If the requested information does not fit into an *AIL_INT64*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetRestore

Synopsis

Restore a 3D metrology context from disk.

Syntax

```
AIL_ID M3dmetRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *Context3dmetIdPtr    //out
)
```

Description

This function restores a 3D metrology context that was previously saved to a file, using M3dmetSave() or M3dmetStream(). This function restores all of the 3D metrology context's settings that were in effect when it was saved.

When the restored 3D metrology context is no longer required, release it using M3dmetFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the 3D metrology context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the 3D metrology context. The function handles (internally) the opening and closing of the file.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, 3D metrology files have an M3DMET extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the 3D metrology context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Context3dmetIdPtr

Specifies the address of the variable in which to write the 3D metrology context identifier or specifies the data type that the function should use to return the 3D metrology context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D metrology context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated 3D metrology context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_3DMET_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the 3D metrology context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the fit 3D metrology context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored fit 3D metrology context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the statistics 3D metrology context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored statistics 3D metrology context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the 3D metrology context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_3DMET_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetSave

Synopsis

Save a 3D metrology context to a file.

Syntax

```
void M3dmetSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID Context3dmetId,       //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about a previously allocated 3D metrology context to disk.

To load a saved context, use either M3dmetRestore() or M3dmetStream().

Parameters

FileName

Specifies the name and path of the file in which to save the 3D metrology context; it is recommended that you use the M3DMET file extension for easier use with other Aurora Imaging software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). The recommended extension is M3DMET. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Context3dmetId

Specifies the identifier of the 3D metrology context to save. The context must have been successfully allocated with M3dmetAlloc() prior to calling this function.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetStat

Synopsis

Compute a variety of statistics on the distances between a point cloud or depth map, and a point cloud, depth map, or 3D geometry.

Syntax

```
void M3dmetStat(
    AIL_ID StatContext3dmetId,       //in
    AIL_ID SrcContainerOrImageBufId, //in
    AIL_ID RefAilObjectId,           //in
    AIL_ID StatResult3dmetId,        //out
    AIL_INT64 DistanceType,          //in
    AIL_INT64 Condition,             //in
    AIL_DOUBLE CondLow,              //in
    AIL_DOUBLE CondHigh,             //in
    AIL_INT64 ControlFlag            //in
)
```

Description

This function calculates a variety of statistics on the distance measurements collected between a source and reference Aurora Imaging Library object.

The specified statistics 3D metrology context establishes which statistics to compute, and the DistanceType and Condition parameters establish the type of distance measurements to use in computing the statistics. Note that you can use a predefined context to calculate a single statistic. In this case, the statistic is calculated using its default settings.

You can use this function to, for example, calculate the mean distance between all points in a point cloud and the center of a 3D sphere geometry object.

Parameters

StatContext3dmetId

Specifies a previously allocated statistics 3D metrology context (used to evaluate multiple statistical calculations), or a predefined statistics 3D metrology context (used to evaluate a single statistical calculation).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the statistics 3D metrology context, or specific statistic to calculate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STAT_CONTEXT_MAX	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_MAX which is set to M_ENABLE. Use this predefined context to calculate the maximum distance between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MAX_ABS	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_MAX_ABS which is set to M_ENABLE. Use this predefined context to calculate the maximum absolute distance between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MEAN	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_MEAN which is set to M_ENABLE. Use this predefined context to calculate the average distance between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MEAN_ABS	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_MEAN_ABS which is set to M_ENABLE. Use this predefined context to calculate the average absolute distance between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_CONTEXT_MIN	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_MIN which is set to M_ENABLE. Use this predefined context to calculate the minimum distance between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MIN_ABS	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_MIN_ABS which is set to M_ENABLE. Use this predefined context to calculate the minimum absolute distance between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_NUMBER	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_NUMBER which is set to M_ENABLE. Use this predefined context to calculate the number of points that meet the condition specified using the Condition, CondLow and CondHigh parameters.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_RMS	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_RMS which is set to M_ENABLE. Use this predefined context to calculate the root-mean-square (RMS) error of the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_STANDARD_DEVIATION	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_STANDARD_DEVIATION which is set to M_ENABLE. Use this predefined context to calculate the standard deviation of all the distances calculated between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_SUM	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_SUM which is set to M_ENABLE. Use this predefined context to calculate the sum of all the distances calculated between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_SUM_ABS	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_SUM_ABS which is set to M_ENABLE. Use this predefined context to calculate the sum of all the absolute distances calculated between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_SUM_OF_SQUARES	Specifies a predefined statistics 3D metrology context with all the control types (M3dmetControl()) in the context set to their default, except for M_STAT_SUM_OF_SQUARES which is set to M_ENABLE. Use this predefined context to calculate the sum of squared distances between the point cloud or depth map, and the specified reference object.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics 3D metrology context identifier	Specifies a statistics 3D metrology context identifier, previously allocated using M3dmetAlloc() with M_STATISTICS_CONTEXT. You can specify which statistics calculations are evaluated using M3dmetControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerOrImageBufId

Specifies the source point cloud container, depth map container, or depth map image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud or depth map

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Depth map container identifier	Specifies the identifier of a depth map container. The container must store data in a 3D-processable depth map format. You can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map. The container can have a region component; only the values inside the region component will be used in the statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer can have region of interest (ROI) in raster format; only the values inside the ROI will be used in the statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies the identifier of a point cloud container. The container must be 3D-processable. You can use MbufInquireContainer() with M_3D_PROCESSABLE to ensure that the container contains a 3D-processable point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

RefAilObjectId

Specifies the identifier of the reference Aurora Imaging Library object, with respect to which distances will be measured.

You can set the RefAilObjectId parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the reference Aurora Imaging Library object identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_XY_PLANE	Specifies the XY (Z=0) plane. > This is equivalent to passing a 3D plane geometry object, and can therefore be used with the same DistanceType parameter settings.	c		e	f	g	a		i		j	k	l	m	n	b
3D geometry object identifier	Specifies the identifier of a 3D geometry object to use as the reference Aurora Imaging Library object. The geometry object must have been previously allocated on the required system using M3dgeoAlloc() and must have been successfully defined. Supported 3D geometries include box, cylinder, line, plane, point, and sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map container identifier	Specifies the identifier of a depth map container. The container must store data in a 3D-processable depth map format. You can use MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP to ensure that the container contains a 3D-processable depth map. The container must not have a region component.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer is allocated using MbufAlloc2d(), and must not have a region of interest (ROI) associated with it.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies the identifier of a point cloud container. The container must be 3D-processable. You can use MbufInquireContainer() with M_3D_PROCESSABLE to ensure that the container contains a 3D-processable point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

See the Parameter associations section for possible values that can be specified.

StatResult3dmetId

Specifies the identifier of the statistics 3D metrology result buffer in which to store the results. The result buffer must have been allocated using M3dmetAllocResult() with M_STATISTICS_RESULT.

DistanceType

Specifies the type of distance to calculate.

See the Parameter associations section for possible values that can be specified.

Condition

Specifies the condition for which distance measurements will be used in statistics calculations.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the condition																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL	Specifies to use all distance measurements.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Specifies to use all distance measurements greater than CondLow.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Specifies to use all distance measurements greater than or equal to CondLow.	c		e	f	g	a		i		j	k	l	m	n	b
M_IN_RANGE	Specifies to use all distance measurements between CondLow and CondHigh (inclusive).	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS	Specifies to use all distance measurements less than CondLow.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL	Specifies to use all distance measurements less than or equal to CondLow.	c		e	f	g	a		i		j	k	l	m	n	b

CondLow

Specifies the lower limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the lower limit of the selected condition																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that this parameter is not applicable.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the lower limit of the condition.	c		e	f	g	a		i		j	k	l	m	n	b

CondHigh

Specifies the upper limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the upper limit of the selected condition																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that this parameter is not applicable.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper limit of the condition.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **RefAilObjectId** and **DistanceType** parameters.

- For specifying the type of distance calculation for the specified reference object

To specify the type of distance measurement, the DistanceType parameter can be set to one of the following values, depending on the type of reference object (RefAilObjectId) being measured against.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of distance calculation for the specified reference object

RefAilObjectId / DistanceType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
3D box geometry object identfier	Specifies a 3D box geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a box.															
M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance to the surface of the box. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_MANHATTAN_DISTANCE_TO_SURFACE	Specifies to calculate the Manhattan distance to the surface of the box. Distance measurements are positive for points outside the box, and negative for points inside the box.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance to the surface of the box. Distance measurements are positive for points outside the box, and negative for points inside the box.	c		e	f	g	a		i		j	k	l	m	n	b
3D cylinder geometry object identifier	Specifies a 3D cylinder geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance to the surface of the cylinder. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_CENTER_AXIS	Specifies to calculate the distance to the cylinder's central axis. The axis extends infinitely in both directions, even if the cylinder has a finite size.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_CENTER_AXIS_SQUARED	Specifies to calculate the square of the distance to the cylinder's central axis. The axis extends infinitely in both directions, even if the cylinder has a finite size.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance to the surface of the cylinder. Distance measurements are positive for points outside the cylinder, and negative for points inside the cylinder.	c		e	f	g	a		i		j	k	l	m	n	b
3D line geometry object identifier	Specifies a 3D line geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a line.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_LINE	Specifies to calculate the distance to the line. The length of the line is respected; if the line is finite, it is not extended infinitely when calculating distances.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_LINE_SQUARED	Specifies to calculate the square of the distance to the line. The length of the line is respected; if the line is finite, it is not extended infinitely when calculating distances.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier	Specifies a 3D plane geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. When the reference object is a 3D plane geometry object, calculations are quicker if the source object is a depth map rather than a point cloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance, perpendicular to the plane. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ABSOLUTE_DISTANCE_Z_TO_SURFACE	Specifies to calculate the absolute distance along the Z-axis to the plane. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance, perpendicular to the plane. Distance measurements are positive for points on the same side as the plane's normal, and negative on the other side.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED_DISTANCE_Z_TO_SURFACE	Specifies to calculate the signed distance along the Z-axis to the plane. Distance measurements are positive if the point is above the plane and negative below.	c		e	f	g	a		i		j	k	l	m	n	b
3D point geometry object identifier		Specifies a 3D point geometry object, previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a point.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_POINT	Specifies to calculate the distance to the point.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_POINT_SQUARED	Specifies to calculate the square of the distance to the point.	c		e	f	g	a		i		j	k	l	m	n	b
3D sphere geometry object identifier		Specifies a 3D sphere geometry object, allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a sphere.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE_DISTANCE_TO_SURFACE	Specifies to calculate the absolute distance to the surface of the sphere. The returned value is always positive.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_CENTER	Specifies to calculate the signed distance to the center of the sphere.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISTANCE_TO_CENTER_SQUARED	Specifies to calculate the square of the distance to the center of the sphere.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED_DISTANCE_TO_SURFACE	Specifies to calculate the signed distance to the surface of the sphere. Distance measurements are positive for points outside the sphere, and negative for points inside the sphere.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map container identifier		Specifies a depth map container, allocated with MbufAllocContainer() with M_PROC. The container must not have a region component. When the source and reference Aurora Imaging Library objects are depth maps, calculations are completed more quickly.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ABSOLUTE_DISTANCE_Z_TO_SURFACE	Specifies to calculate the absolute distance to the position in the depth map that is directly above or below the source along the Z-axis. The returned value is always positive. If a point would be projected outside of the depth map, the point is not used in statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED_DISTANCE_Z_TO_SURFACE	Specifies to calculate the signed distance using the Z-coordinate of the corresponding pixel of the reference depth map. That is, the distance is calculated along the Z-axis. Distance measurements are positive for points above the depth map, and negative for points below the depth map. If a point would be projected outside of the depth map, the point is not used in statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b

Depth map image buffer identifier	Specifies the identifier of a depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer. It must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). The image buffer must not have a region of interest (ROI) associated with it. When the source and reference Aurora Imaging Library objects are depth maps, calculations are completed more quickly. The speed is further increased if both depth maps use the same calibration context.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE_DISTANCE_Z_TO_SURFACE	Specifies to calculate the absolute distance to the position in the depth map that is directly above or below the source along the Z-axis. The returned value is always positive. If a point would be projected outside of the depth map, the point is not used in statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED_DISTANCE_Z_TO_SURFACE	Specifies to calculate the signed distance using the Z-coordinate of the corresponding pixel of the reference depth map. That is, the distance is calculated along the Z-axis. Distance measurements are positive for points above the depth map, and negative for points below the depth map. If a point would be projected outside of the depth map, the point is not used in statistics calculations.	c		e	f	g	a		i		j	k	l	m	n	b
Point cloud container identifier	Specifies a point cloud container, allocated with MbufAllocContainer() with M_PROC. The function runs much slower when the reference Aurora Imaging Library object is a point cloud container. If speed is an issue, you can specify a range for calculating distance measurements using the Condition, CondHigh and CondLow parameters. Any distance measurements outside the specified range are not used in statistics calculations. You can also project the reference point cloud into a fully corrected depth map first (using M3dimProject()), at the cost of precision.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_MESH	Specifies to calculate the shortest distance to the reference object's mesh. This calculation requires that the reference object have a mesh component.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TO_NEAREST_NEIGHBOR	Specifies to calculate the distance to the nearest point in the reference point cloud.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetStream

Synopsis

Load, restore, or save a 3D metrology context from/to a file or memory stream.

Syntax

```
void M3dmetStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *Context3dmetIdPtr,     //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a 3D metrology context from/to a file or memory stream.

To inquire the number of bytes necessary to save a 3D metrology context to a memory stream, you should first call this function (**M3dmetStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using M3dmetSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

Using **M3dmetStream**(), you can choose to save a backwards-compatible version of the 3D metrology context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore 3D metrology contexts saved using any Aurora Imaging Library version with which this function was available. Settings that do not exist in the lower version will be filled with default values when the 3D metrology context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the 3D metrology context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the 3D metrology context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b

M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the 3D metrology context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Context3dmetIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the 3D metrology context. If the M_RESTORE operation is not successful, **M_NULL** is returned.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the 3D metrology context, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of the 3D metrology context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **Context3dmetIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
Context3dmetIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a 3D metrology context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the 3D metrology context must be compatible when saved. The size of a 3D metrology context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b

M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
Context3dmetIdPtr	Specifies the address of the variable containing the identifier of the 3D metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated 3D metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the 3D metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. 3D metrology contexts typically have an M3DMET file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
Context3dmetIdPtr	Specifies the address of the variable containing the identifier of the 3D metrology context. All previous settings are overwritten with those of the loaded 3D metrology context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the 3D metrology context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores a 3D metrology context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to restore the 3D metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. 3D metrology contexts typically have an M3DMET file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		Specifies the system on which to allocate the 3D metrology context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
		System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	Context3dmetIdPtr		Specifies the address of the variable in which to write the identifier of the 3D metrology context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE			Saves a 3D metrology context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a 3D metrology context to a file, use the M3DMET file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		Specifies the Aurora Imaging Library version with which the 3D metrology context must be compatible when saved.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b

	Context3dmetIdPtr	Specifies the address of the variable containing the identifier of the 3D metrology context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetVolume

Synopsis

Computes the volume of a point cloud's mesh or of a depth map, delimited optionally by a specified reference object.

Syntax

```
AIL_DOUBLE M3dmetVolume(
    AIL_ID SrcContainerOrImageBufId,   //in
    AIL_ID Reference3dgeoOrImageBufId, //in
    AIL_INT64 Options,                 //in
    AIL_INT64 ControlFlag,             //in
    AIL_DOUBLE *VolumePtr,             //out
    AIL_INT64 *StatusPtr               //out
)
```

Description

This function calculates the volume of a point cloud's mesh or of a depth map, delimited optionally by a specified reference object.

If the source Aurora Imaging Library object is not sufficient to define a closed volume, you must specify a reference object. This function extends the boundaries of the source Aurora Imaging Library object (and for a mesh, its holes) onto the reference Aurora Imaging Library object to define the closed 3D shape whose volume to calculate. How the boundaries extend onto the reference Aurora Imaging Library object depends on the type of the source Aurora Imaging Library object. If the boundaries of a mesh are extended, they will be extended perpendicular to the reference Aurora Imaging Library object. If the boundaries of a depth map are extended, they will be extended in the Z-direction until they meet the reference Aurora Imaging Library object.

Even if not required, you can specify a reference object to limit the volume to calculate. You can use the Options parameter to specify how the position of the reference Aurora Imaging Library object, relative to enclosed shape, defines the volume calculation. For example, if you use a 3D plane geometry as the reference object of a mesh, you can specify whether to calculate the volume of the mesh above or below the plane.

Parameters

SrcContainerOrImageBufId

Specifies the point cloud or depth map for which to calculate the volume.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud or depth map																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Depth map container identifier	Specifies the identifier of the container, containing a 3D-processable depth map, to use as the source Aurora Imaging Library object. The container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). Any invalid points in the depth map are treated like holes, but you can use M3dimFillGaps() to fill them before using this function. The function will compute the volume of the enclosed shape defined by the depth map, a reference object, and planes defined by the extension of the edges of the depth map to the reference Aurora Imaging Library object in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of the depth map image buffer to use as the source Aurora Imaging Library object. The image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned buffer and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). This image buffer can have a region of interest (ROI) associated with it, in raster format (M_RASTER or M_VECTOR_AND_RASTER). Only points inside the ROI will be used in the volume calculation. Any invalid points in the depth map are treated like holes, but you can use M3dimFillGaps() to fill them before using this function. The function will compute the volume of the enclosed shape defined by the depth map, a reference object, and planes defined by the extension of the edges of the depth map to the reference Aurora Imaging Library object in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b

Meshed point cloud container identifier	Specifies the identifier of the container, containing a 3D-processable meshed point cloud, to use as the source Aurora Imaging Library object. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must contain a mesh component. If the mesh does not contain any holes, the function computes the volume of the mesh, optionally delimited by the reference object. If the mesh contains holes, the function will compute the volume of the enclosed shape defined by the mesh, the reference object, and the planes defined by the extension of the edges of the mesh to the reference object at a perpendicular angle. For more detailed information on how the volume is calculated for meshes with holes, see the Calculating a volume section of Chapter 39: 3D metrology.	c		e	f	g	a		i		j	k	l	m	n	b

Reference3dgeoOrImageBufId

Specifies the reference Aurora Imaging Library object to use to define a closed 3D shape whose volume to calculate.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the reference Aurora Imaging Library object																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to perform the default behavior. If the source Aurora Imaging Library object is a point cloud container, the default behavior is the same as M_NULL. If the source Aurora Imaging Library object is a depth map, the default behavior is the same as M_XY_PLANE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to not use a reference Aurora Imaging Library object for defining the volume to calculate. M_NULL can only be specified when calculating the volume of meshes without any holes.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_PLANE	Specifies to use the XY (Z=0) plane as the reference Aurora Imaging Library object. If the source Aurora Imaging Library object is a point cloud container, holes in the mesh are allowed, and the boundaries of the holes in the mesh are extended to the XY-plane at an angle perpendicular to the plane. If the source Aurora Imaging Library object is a depth map, the boundaries of the holes in the depth map are extended in the Z-direction to the XY-plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier	Specifies the identifier of the 3D plane geometry object to use as the reference Aurora Imaging Library object. The 3D plane geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. If the source Aurora Imaging Library object is a point cloud container, holes in the mesh are allowed, and the boundaries of the holes in the mesh are extended to the reference plane at an angle perpendicular to the plane. If the source Aurora Imaging Library object is a depth map, the boundaries of the holes in the depth map are extended in the Z-direction to the reference plane.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map container identifier	Specifies the identifier of the depth map container to use as the reference Aurora Imaging Library object. This is only usable with a depth map as the source Aurora Imaging Library object. The container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). When both source and reference Aurora Imaging Library objects are depth maps, their calibration information can vary in Z, but not in X and Y. That is, the depth map specified with SrcContainerOrImageBufId must have equivalent calibration information for X and Y as the depth map specified with Reference3dgeoOrImageBufId. The operation is performed point-to-point; therefore, each respective pixel must represent the same area in the real world in X and Y.	c		e	f	g	a		i		j	k	l	m	n	b

Depth map image buffer identifier	Specifies the identifier of the depth map image buffer to use as the reference Aurora Imaging Library object. This is only usable with a depth map as the source Aurora Imaging Library object. This image buffer can have a region of interest (ROI) associated with it, in raster format (M_RASTER or M_VECTOR_AND_RASTER). When both source and reference Aurora Imaging Library objects are depth maps, their calibration information can vary in Z, but not in X and Y. That is, the depth map specified with SrcContainerOrImageBufId must have equivalent calibration information for X and Y as the depth map specified with Reference3dgeoOrImageBufId. The operation is performed point-to-point; therefore, each respective pixel must represent the same area in the real world in X and Y.	c		e	f	g	a		i		j	k	l	m	n	b

Options

Specifies how to calculate the total volume, given the position of the source Aurora Imaging Library object relative to the reference object. If you are calculating the volume of a point cloud without any reference Aurora Imaging Library object, set this parameter to M_DEFAULT.

If the reference object is a plane, any volume on the same side as the plane's normal is considered to be above the plane.

If the reference object is a depth map, a given pixel in the source depth map (at a certain X- and Y-coordinate) is considered above a pixel (at the same X- and Y-coordinate) in the reference depth map if its Z-coordinate is greater.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the condition																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_TOTAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABOVE	Specifies that the total volume is equal only to the volume above the reference Aurora Imaging Library object. Any volume below the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIFFERENCE	Specifies that the total volume is calculated as the volume above the reference Aurora Imaging Library object, minus the volume below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOTAL	Specifies that the total volume is calculated as the sum of all volumes between the source and reference Aurora Imaging Library objects, whether above or below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNDER	Specifies that the total volume is equal only to the volume under the reference Aurora Imaging Library object. Any volume above the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

VolumePtr

Specifies the address of the variable in which to write the calculated volume. Since this function also returns the volume, this parameter can be set to **M_NULL**.

StatusPtr

Specifies the address of the variable in which to write the status of the volume calculation.

If not required, set this parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the status of the volume operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FAIL_GAPS	Specifies that the volume was not successfully computed, since the source Aurora Imaging Library object was a mesh with gaps, and there was no reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b

M_FAIL_INVALID_MESH	Specifies that the volume was not successfully computed, since the mesh was not usable.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies that the volume was successfully computed.	c		e	f	g	a		i		j	k	l	m	n	b
M_WARNING_GAPS	Specifies that the volume was successfully computed, but there were gaps in the specified source depth map. The gaps were ignored in the volume calculation.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the computed volume, if successful. If the volume calculation fails, **M_NULL** is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.

M3dmetVolumeEx

Synopsis

Computes the volume of a point cloud's mesh or of a depth map, delimited optionally by a specified reference object, and stores the result in a result buffer.

Syntax

```
void M3dmetVolumeEx(
    AIL_ID Volume3dmetContextId,       //in
    AIL_ID SrcContainerOrImageBufId,   //in
    AIL_ID Reference3dgeoOrImageBufId, //in
    AIL_ID Calculate3dmetResultId,     //out
    AIL_INT64 ControlFlag              //in
)
```

Description

This function calculates the volume of a point cloud's mesh or of a depth map, delimited optionally by a specified reference object, and stores the result in a result buffer. To retrieve the result and related calculation information, use M3dmetGetResult(). If you only need the calculated volume, you can use M3dmetVolume() instead of **M3dmetVolumeEx**().

You can specify a predefined context for calculating the volume.

If the source Aurora Imaging Library object is not sufficient to define a closed volume, you must specify a reference object. This function extends the boundaries of the source Aurora Imaging Library object (and for a mesh, its holes) onto the reference Aurora Imaging Library object to define the closed 3D shape whose volume to calculate. How the boundaries extend onto the reference Aurora Imaging Library object depends on the type of the source Aurora Imaging Library object. If the boundaries of a mesh are extended, they will be extended perpendicular to the reference Aurora Imaging Library object. If the boundaries of a depth map are extended, they will be extended in the Z-direction until they meet the reference Aurora Imaging Library object.

Even if not required, you can specify a reference object that limits the volume to calculate. Use M3dmetControl() with M_VOLUME_MODE to specify how the position of the reference Aurora Imaging Library object, relative to the enclosed shape, defines the volume calculation. For example, if you use a 3D plane geometry as the reference object of a mesh, you can specify whether to calculate the volume of the mesh above or below the plane.

Parameters

Volume3dmetContextId

Specifies the context for volume calculation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the volume 3D metrology context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default volume 3D metrology context of the current Aurora Imaging Library application. > The volume operation will use default values for all volume control types listed in M3dmetControl().	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_CONTEXT_ABOVE	Specifies a predefined volume 3D metrology context with all volume context control types (M3dmetControl()) set to their default, except M_VOLUME_MODE which is set to M_ABOVE. Use this predefined context to specify that the total volume is equal only to the volume above the reference Aurora Imaging Library object. Any volume below the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_CONTEXT_DIFFERENCE	Specifies a predefined volume 3D metrology context with all volume context control types (M3dmetControl()) set to their default, except M_VOLUME_MODE which is set to M_DIFFERENCE. Use this predefined context to calculate the total volume as the volume above the reference Aurora Imaging Library object, minus the volume below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b

M_VOLUME_CONTEXT_TOTAL	Specifies a predefined volume 3D metrology context with all volume context control types (M3dmetControl()) set to their default, except M_VOLUME_MODE which is set to M_TOTAL. Use this predefined context to calculate the total volume as the sum of all volumes between the source and reference Aurora Imaging Library objects, whether above or below the reference Aurora Imaging Library object.	c		e	f	g	a		i		j	k	l	m	n	b
M_VOLUME_CONTEXT_UNDER	Specifies a predefined volume 3D metrology context with all volume context control types (M3dmetControl()) set to their default, except M_VOLUME_MODE which is set to M_UNDER. Use this predefined context to specify that the total volume is equal only to the volume under the reference Aurora Imaging Library object. Any volume above the reference Aurora Imaging Library object is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
Volume 3D metrology context identifier	Specifies the identifier of a volume 3D metrology context, previously allocated using M3dmetAlloc() with M_VOLUME_CONTEXT. > If a previously allocated context is specified, the function applies the volume control settings specified using M3dmetControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerOrImageBufId

Specifies the point cloud or depth map for which to calculate the volume.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the point cloud or depth map																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Depth map container identifier	Specifies the identifier of the container, containing a 3D-processable depth map, to use as the source Aurora Imaging Library object. The container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). Any invalid points in the depth map are treated like holes, but you can use M3dimFillGaps() to fill them before using this function. The function will compute the volume of the enclosed shape defined by the depth map, a reference object, and planes defined by the extension of the edges of the depth map to the reference Aurora Imaging Library object in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of the depth map image buffer to use as the source Aurora Imaging Library object. The image buffer must be a 1-band, 8-bit, 16-bit or 32-bit unsigned buffer and must contain a fully corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). This image buffer can have a region of interest (ROI) associated with it, in raster format (M_RASTER or M_VECTOR_AND_RASTER). Only points inside the ROI will be used in the volume calculation. Any invalid points in the depth map are treated like holes, but you can use M3dimFillGaps() to fill them before using this function. The function will compute the volume of the enclosed shape defined by the depth map, a reference object, and planes defined by the extension of the edges of the depth map to the reference Aurora Imaging Library object in the Z-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Meshed point cloud container identifier	Specifies the identifier of the container, containing a 3D-processable meshed point cloud, to use as the source Aurora Imaging Library object. The container must be 3D-processable (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE, the function returns M_PROCESSABLE). The container must contain a mesh component. If the mesh does not contain any holes, the function computes the volume of the mesh, optionally delimited by the reference object. If the mesh contains holes, the function will compute the volume of the enclosed shape defined by the mesh, the reference object, and the planes defined by the extension of the edges of the mesh to the reference object at a perpendicular angle. For more detailed information on how the volume is calculated for meshes with holes, see the Calculating a volume section of Chapter 39: 3D metrology.	c		e	f	g	a		i		j	k	l	m	n	b

Reference3dgeoOrImageBufId

Specifies the reference Aurora Imaging Library object to use to define a closed 3D shape whose volume to calculate.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the reference Aurora Imaging Library object																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to perform the default behavior. If the source Aurora Imaging Library object is a point cloud container, the default behavior is the same as M_NULL. If the source Aurora Imaging Library object is a depth map, the default behavior is the same as M_XY_PLANE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to not use a reference Aurora Imaging Library object for defining the volume to calculate. M_NULL can only be specified when calculating the volume of meshes without any holes.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_PLANE	Specifies to use the XY (Z=0) plane as the reference Aurora Imaging Library object. If the source Aurora Imaging Library object is a point cloud container, holes in the mesh are allowed, and the boundaries of the holes in the mesh are extended to the XY-plane at an angle perpendicular to the plane. If the source Aurora Imaging Library object is a depth map, the boundaries of the holes in the depth map are extended in the Z-direction to the XY-plane.	c		e	f	g	a		i		j	k	l	m	n	b
3D plane geometry object identifier	Specifies the identifier of the 3D plane geometry object to use as the reference Aurora Imaging Library object. The 3D plane geometry object must have been previously allocated using M3dgeoAlloc() with M_GEOMETRY, and defined as a plane. If the source Aurora Imaging Library object is a point cloud container, holes in the mesh are allowed, and the boundaries of the holes in the mesh are extended to the reference plane at an angle perpendicular to the plane. If the source Aurora Imaging Library object is a depth map, the boundaries of the holes in the depth map are extended in the Z-direction to the reference plane.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map container identifier	Specifies the identifier of the depth map container to use as the reference Aurora Imaging Library object. This is only usable with a depth map as the source Aurora Imaging Library object. The container must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). When both source and reference Aurora Imaging Library objects are depth maps, their calibration information can vary in Z, but not in X and Y. That is, the depth map specified with SrcContainerOrImageBufId must have equivalent calibration information for X and Y as the depth map specified with Reference3dgeoOrImageBufId. The operation is performed point-to-point; therefore, each respective pixel must represent the same area in the real world in X and Y.	c		e	f	g	a		i		j	k	l	m	n	b
Depth map image buffer identifier	Specifies the identifier of the depth map image buffer to use as the reference Aurora Imaging Library object. This is only usable with a depth map as the source Aurora Imaging Library object. This image buffer can have a region of interest (ROI) associated with it, in raster format (M_RASTER or M_VECTOR_AND_RASTER). When both source and reference Aurora Imaging Library objects are depth maps, their calibration information can vary in Z, but not in X and Y. That is, the depth map specified with SrcContainerOrImageBufId must have equivalent calibration information for X and Y as the depth map specified with Reference3dgeoOrImageBufId. The operation is performed point-to-point; therefore, each respective pixel must represent the same area in the real world in X and Y.	c		e	f	g	a		i		j	k	l	m	n	b

Calculate3dmetResultId

Specifies the identifier of the calculate 3D metrology result buffer in which to store the results of the volume operation.

The result buffer must have been previously allocated using M3dmetAllocResult() with M_CALCULATE_RESULT. To retrieve results for the volume operation, use M3dmetGetResult().

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ail3dmet11.lib.
DLL	Requires ail11.dll; ail3dmet11.dll.